Ensino que te conecta

BACHARELADO EM

# ENGENHARIA DE SOFTWARE

## ACCELERATION OF PATTERN MATCHING ALGORITHMS USING PARALLEL PROGRAMMING

MATHEUS ANTÔNIO DE CASTRO DE BARROS

Brasília - DF, 2026

**MATHEUS ANTÔNIO DE CASTRO DE BARROS**

# ACCELERATION OF PATTERN MATCHING ALGORITHMS USING PARALLEL PROGRAMMING

Undergraduate Thesis submitted in partial fulfillment of the requirements for the degree of Bachelor in Software Engineering at the Brazilian Institute of Education, Development and Research (IDP).

**Advisor**
Jeremias Moreira Gomes

Brasília - DF, 2026

Código de catalogação na publicação – CIP

B277a Barros, Matheus Antônio de Castro de
Acceleration of Pattern Matching Algorithms Using Parallel Programming / Matheus Antônio de Castro de Barros. — Brasília: Instituto Brasileiro de Ensino, Desenvolvimento e Pesquisa, 2026.

109 f.: il.

Orientador: Prof. Dr. Jeremias Moreira Gomes

Monografia (Graduação em Engenharia de Software). — Instituto Brasileiro de Ensino, Desenvolvimento e Pesquisa – IDP, 2026.

1. Aho–Corasick. 2. Parallel programming. 3. Posix threads. I. Título

CDD 005.10218

Elaborada por Biblioteca Ministro Moreira Alves

**MATHEUS ANTÔNIO DE CASTRO DE BARROS**

# ACCELERATION OF PATTERN MATCHING ALGORITHMS USING PARALLEL PROGRAMMING

Undergraduate Thesis submitted in partial fulfillment of the requirements for the degree of Bachelor in Software Engineering at the Brazilian Institute of Education, Development and Research (IDP).

Approved on July 8, 2026

**Examining Committee**

Jeremias Moreira Gomes (Advisor) - IDP

Thálisson de Oliveira Lopes (Committee Member) - IDP

Caio César Rodrigues Garcez (Committee Member) - IDP

*À minha mãe, que sempre acreditou em mim.*

# AGRADECIMENTOS

Agradeço à minha família, pelo incentivo do início ao fim da graduação; ao meu orientador, Jeremias Moreira Gomes, pela excelência e rigor ao longo deste trabalho; e à minha noiva Marina, por me apoiar e orar por mim.

# ABSTRACT

The Aho-Corasick algorithm is a cornerstone of signature-based network intrusion detection systems and malware analysis tools because it scans an input in linear time while matching many patterns simultaneously. Its canonical automaton traversal is sequential, however, leaving modern multi-core processors underused when a single large input must be inspected. This work designs, implements, and evaluates data-parallel Aho-Corasick scanning strategies built on POSIX Threads for shared-memory multi-core CPUs. The input text is partitioned into overlapping segments that worker threads scan concurrently over a shared, strictly read-only automaton, keeping the search path free of locks; a flattened, contiguous output layout is also proposed to reduce the pointer-chasing cost of emitting matches. The evaluation targets the memory-bound regime in which realistic Snort and Emerging Threats rule sets produce automata up to roughly $507$ MiB, far beyond the effective last-level cache capacity. On a homogeneous AMD Ryzen 9 9950X workstation (Zen 5, sixteen cores and thirty-two hardware threads), the dynamic flat strategy reaches $22.91\times$ speedup and $7{,}542$ MB/s on the full Snort dictionary, and $18.96\times$ speedup and $3{,}979$ MB/s on the larger Emerging Threats dictionary. The same strategy remains insensitive to corpus content for the Snort workload, reaching $7{,}976$ MB/s on SimpleWiki and $7{,}595$ MB/s on Enron, while the coarsest tested dynamic-task granularity reaches $7{,}804$ MB/s. End-to-end accelerations over the eight-fold Enron corpus reach $22.23\times$ for Snort and $17.71\times$ for Emerging Threats, showing that construction remains a bounded overhead and that search dominates the pipeline. A separate controlled diagnostic on content-skewed corpora — identical bytes and total matches, concentrated in one region of the text — confirms the load-balancing mechanism behind the dynamic strategy: static partitions lose up to roughly $30\%$ median throughput, while dynamic dispatch remains balanced and overtakes them. The central finding is that, once the automaton exceeds the effective cache capacity, Aho-Corasick scaling is limited primarily by the memory subsystem rather than by the matching logic; text parallelism over a shared automaton, combined with flat match emission, is the most effective response in this regime.

**Keywords:** Aho–Corasick, parallel programming, parallel computing, multi-pattern matching, POSIX threads.

# RESUMO

O algoritmo Aho-Corasick é um pilar dos sistemas de detecção de intrusão de rede baseados em assinaturas e das ferramentas de análise de malware, pois varre uma entrada em tempo linear enquanto casa muitos padrões simultaneamente. Sua travessia canônica do autômato, porém, é sequencial e subutiliza os processadores multinúcleo modernos quando uma única entrada de grande porte precisa ser inspecionada. Este trabalho projeta, implementa e avalia estratégias de varredura Aho-Corasick com paralelismo de dados, construídas sobre POSIX Threads para CPUs multinúcleo de memória compartilhada. O texto de entrada é particionado em segmentos sobrepostos que as threads varrem concorrentemente sobre um autômato compartilhado e estritamente somente-leitura, mantendo a busca livre de travas; propõe-se também um layout de saída achatado e contíguo que reduz o custo de perseguição de ponteiros na emissão de casamentos. A avaliação foca o regime limitado por memória, no qual conjuntos de regras realistas do Snort e do Emerging Threats produzem autômatos de até cerca de $507$ MiB, muito além da capacidade efetiva da cache de último nível. Em uma estação de trabalho homogênea AMD Ryzen 9 9950X (Zen 5, dezesseis núcleos, trinta e duas threads de hardware), a estratégia dinâmica com saída achatada atinge $22{,}91\times$ de speedup e $7\,542$ MB/s no dicionário completo do Snort, e $18{,}96\times$ e $3\,979$ MB/s no do Emerging Threats. A mesma estratégia permanece insensível ao conteúdo do corpus na carga do Snort, com $7\,976$ MB/s no SimpleWiki e $7\,595$ MB/s no Enron, e a granularidade de tarefa mais grossa testada atinge $7\,804$ MB/s. De ponta a ponta, sobre o corpus Enron replicado oito vezes, as acelerações atingem $22{,}23\times$ para o Snort e $17{,}71\times$ para o Emerging Threats, mostrando que a construção permanece um custo limitado e que a busca domina o pipeline. Um diagnóstico controlado sobre corpora com conteúdo enviesado — mesmos bytes e mesmo total de casamentos, concentrados em uma região do texto — confirma o balanceamento de carga da estratégia dinâmica: partições estáticas perdem até cerca de $30\%$ as ultrapassa. A conclusão central é que, quando o autômato excede a capacidade efetiva da cache, a escalabilidade do Aho-Corasick passa a ser limitada pelo subsistema de memória, e não pela lógica de casamento; o paralelismo de texto sobre um autômato compartilhado, combinado à emissão achatada de casamentos, é a resposta mais eficaz nesse regime.

**Palavras-chave:** Aho–Corasick, Programação Paralela, Casamento de Múltiplos Padrões, POSIX Threads, Detecção de Intrusão.

# LIST OF FIGURES


1 Ideal vs. Amdahl's theoretical speedup ($s = 0.1$). ..... 10

2 Example trie storing {"car", "cat", "cart", "dog"}. Nodes marking the end of a complete word are shown with a double circle. 13

3 Complete Aho–Corasick automaton for the pattern set $K = \{\text{"he"}, \text{"she"}, \text{"hers"}\}$, showing solid blue `goto` transitions, dashed red `fail` links, double-circled accepting states, and their corresponding `output` match sets. ..... 19

4 Flowchart of the Aho–Corasick matching phase. ..... 20

5 Parallel execution pipeline of the proposed framework, divided into three phases: sequential construction of the read-only automaton (Phase 1), concurrent parallel search across text segments (Phase 2), and sequential deterministic merge of match lists (Phase 3). ..... 44

6 Data partitioning and boundary overlap scanning layout, showing matching ownership and warm-up margins across Thread 0 and Thread 1. ..... 45

7 Partitioning policies on heterogeneous cores. Bar length represents elapsed execution time, while labels indicate assigned text size or task identity: (a) static homogeneous chunking leaves the faster Performance core idle at the barrier (stragglers); (b) topology-aware, frequency-weighted chunking sizes each segment to its core's speed; (c) dynamic dispatch hands out small tasks on demand. Both (b) and (c) equalize finishing times. ..... 47

8 Comparison of match-emission layouts: (a) canonical linked outputs using pointer-chasing across scattered memory nodes, and (b) proposed flattened output layout storing contiguous pattern indices. ..... 48
9 Level-synchronous parallel construction of the failure function. Threads compute the failure links of all states at one breadth-first level in parallel, reading only already-finalized links from shallower levels (green), and meet at a barrier before advancing to the next level. ..... 49
10 Text-reading traffic in the two partitioning designs: (a) pure text chunking reads each input byte once while workers share the full automaton; (b) dictionary sharding builds $K$ independent sub-automata and scans the input once per shard, reducing each automaton footprint while multiplying text traffic. ..... 50
11 Cache-residency regimes governing throughput: (a) a small automaton fits within the shared LLC, so transitions are likely to be served from cache; (b) a memory-bound automaton ($\gg$LLC) is likely to produce more cache misses and place sustained pressure on the memory bus, flattening the scaling curve. ..... 67

12 Search throughput as a function of the automaton's memory footprint, on the canonical workstation, at both ends of the thread budget. Each point is one dictionary—Snort-100, Snort-1k, full Snort, and Emerging Threats—so the corpus is held fixed and only the automaton size varies, from 1.9 to 507 MiB. Throughput falls monotonically on both curves as the automaton grows: the single thread drops from 629 to 210 MB/s, and the thirty-two-thread champion (`pthread_dynamic_flat`) collapses from 16,230 to 4,046 MB/s. The dashed line marks the aggregate 64 MiB last-level cache (two 32 MiB slices); the full Snort automaton (55 MiB) already exceeds a single slice, and Emerging Threats (507 MiB) is about eight times the whole cache, so a growing share of state transitions must be served from main memory. Both axes are logarithmic. ..... 68

13 Speedup of text parallelism versus thread count for the winning strategy (`pthread_dynamic_flat`) on the canonical Enron corpus, for a moderate dictionary (Snort, 55 MiB) and a memory-bound dictionary (Emerging Threats, 507 MiB). Scaling is monotonic to the full thirty-two hardware threads; the sixteen-thread mark separates the physical cores from the SMT siblings. The per-point coefficient of variation stays below about 2% bars would fall within the markers and are omitted..... 69

14 Parallel efficiency—speedup divided by thread count (Eq. 4)—for the winning strategy (`pthread_dynamic_flat`) on the canonical Enron corpus, computed against the flat sequential baseline (`sequential_flat`) so that the emission-layout gain is not credited to parallelism. While the speedup itself rises monotonically to $T = 32$ (Figure 13), the efficiency view exposes the saturation of the parallel gain: each added thread contributes less, and the largest single loss occurs when the sweep crosses the dashed line at sixteen threads and the first SMT siblings begin to share physical cores. The gap between the two regimes widens with the thread count, reaching about thirteen percentage points at $T = 32$, consistent with the memory-bandwidth contention discussed above. . . . . 70

# LIST OF TABLES


1 Search results and filters applied per database. ..... 26

2 Pattern dictionaries, the resulting automaton footprint, and their experimental role. ..... 55

3 Text corpora used as scanning input. ..... 55

4 Evaluated strategies as orthogonal design axes; the work-distribution options are alternatives, while the emission and construction axes compose with any of them. ..... 57

5 Per-phase repetition protocol of the formal sweep, verified against `sweep.db`. ..... 59

6 Hypotheses, the experiment that tests each one, and where the outcome is reported. ..... 62

7 Single-threaded scanning throughput as the automaton footprint grows (corpus fixed, single thread). ..... 66

8 Single-threaded gain of the flattened output layout, by automaton footprint. ..... 71

9 Throughput and iteration-to-iteration stability of the text-parallel strategies (thirty-two threads, Snort, canonical corpus). ..... 72

10 Effect of task granularity on the dynamic scheduler ..... 73

11 Parallel automaton construction: build speedup versus the sequential build, by thread count. ..... 74

12 Dictionary sharding (by prefix) across shard counts, compared to text parallelism (Snort, canonical corpus). ..... 75

13 Two-dimensional composition versus the canonical dynamic-flat text-chunking strategy (thirty-two threads). ..... 76

14 Iteration-to-iteration coefficient of variation of the work-distribution strategies on a heterogeneous versus a homogeneous processor (per-thread instrumentation, Snort workload, each machine at its full logical thread count). ..... 77
15 Median search throughput on the content-skew corpus pair (i5-1235U, twelve threads, medians of five independent invocations)..... 79
16 Worker balance under content skew (Snort dictionary, i5-1235U, twelve threads; medians of five independent invocations)..... 79
17 Aggregating views of the measurement database. ..... 91

# CONTENTS


**1 Introduction . . . . . 2**

1.1 Motivation . . . . . 3

1.2 Objectives . . . . . 3

1.3 Structure of the Thesis . . . . . 4

**2 Theoretical Framework . . . . . 6**

2.1 Fundamentals of Parallelism . . . . . 6

*2.1.1 Parallel Computing Architectures . . . . . 6*

*2.1.2 Processes vs Threads . . . . . 7*

*2.1.3 POSIX Thread Model (Pthreads) . . . . . 7*

*2.1.4 Cache Architecture and Memory-Bound Parallelism . . . . . 8*

*2.1.5 Hybrid P/E-Core Architectures . . . . . 9*

2.2 Parallel Performance Evaluation Metrics . . . . . 9

*2.2.1 Execution Time and Overhead . . . . . 9*

*2.2.2 Speedup . . . . . 10*

*2.2.3 Efficiency . . . . . 11*

*2.2.4 Scalability . . . . . 11*

2.3 Tries (Prefix Trees) . . . . . 11

*2.3.1 Definition and Basic Structure . . . . . 12*

*2.3.2 Core Operations . . . . . 12*

*2.3.3 Advantages and Performance Characteristics . . . . . 13*

2.4 Pattern Matching . . . . . 14

*2.4.1 The Pattern Matching Problem . . . . . 14*

*2.4.2 The Naive Algorithm . . . . . 15*

*2.4.3 Knuth-Morris-Pratt (KMP) Algorithm . . . . . 16*

*2.4.4 Boyer-Moore (BM) Algorithm . . . . . 16*

*2.4.5 The Need for Specialized Multi-Pattern Algorithms . . . . . 17*

2.5 The Aho-Corasick Algorithm . . . . . 17

*2.5.1 Context . . . . . 17*

*2.5.2 Algorithm Components and Construction . . . . . 18*

*2.5.3 Matching Phase . . . . . 19*

*2.5.4 Time and Space Complexity . . . . . 21*

*2.5.5 NFA versus DFA Representations . . . . . 21*

*2.5.6 Applications and Relevance . . . . . 22*

**3 Systematic Review ..... 24**
3.1 Research Questions ..... 24
3.2 Search Strategy and Study Selection Protocol ..... 24
*3.2.1Data Sources ..... 24*
*3.2.2Search Queries ..... 25*
*3.2.3Inclusion (IC) and Exclusion (EC) Criteria ..... 25*
*3.2.4Selection Process and Initial Search Outcomes ..... 26*
3.3 Review of Selected Studies ..... 27
*3.3.1A Parallel Algorithm of String Matching Based on Message Passing Interface for Multicore Processors ..... 27*
*3.3.2A Flexible Pattern-Matching Algorithm for Network Intrusion Detection Systems Using Multi-Core Processors ..... 28*
*3.3.3An Optimized Parallel Failure-less Aho-Corasick Algorithm for DNA Sequence Matching ..... 29*
*3.3.4The Diversification and Enhancement of an IDS Scheme for the Cybersecurity Needs of Modern Supply Chains ..... 30*
*3.3.5Fast String Searching on PISA ..... 31*
*3.3.6Practical Multi-Pattern Matching Approach for Fast and Scalable Log Abstraction ..... 33*
*3.3.7Pattern Matching in YARA: Improved Aho-Corasick Algorithm. ..... 34*
*3.3.8SIMD-Accelerated Regular Expression Matching ..... 36*
*3.3.9Harry: A Scalable SIMD-based Multi-literal Pattern Matching Engine for Deep Packet Inspection ..... 37*
*3.3.10Characterizing Realistic Signature-based Intrusion Detection Benchmarks ..... 38*
3.4 Synthesis and Identified Gap ..... 40
**4 Proposed Solution ..... 43**
4.1 Data Partitioning and Boundary Overlap ..... 43
4.2 Load Balancing on Heterogeneous Cores ..... 45
4.3 Flattening the Match-Emission Path ..... 46
4.4 Partitioning the Dictionary ..... 49
**5 Methodology ..... 53**
5.1 Experimental Environment ..... 53
5.2 Pattern Dictionaries and Text Corpora ..... 54
5.3 Execution Model and Invariants ..... 56
5.4 Evaluated Strategies ..... 57

5.5 Metrics and Instrumentation . . . . . 58
5.6 Measurement Protocol . . . . . 59
5.7 Correctness Validation . . . . . 60
5.8 Experimental Design and Hypotheses . . . . . 61

**6 Results and Discussion . . . . . 65**

6.1 Impact of the Automaton Footprint on Throughput . . . . . 66
6.2 Scalability of Text Parallelism . . . . . 67
6.3 The Flattened Output Layout . . . . . 71
6.4 Load Balance and Run-to-Run Stability . . . . . 72
6.5 Task Granularity of the Dynamic Scheduler . . . . . 73
6.6 Parallel Construction of the Automaton . . . . . 73
6.7 Partitioning the Dictionary: A Qualified Negative Result . . . . . 75
6.8 Combined Strategy Evaluation . . . . . 75
6.9 Load Imbalance on Heterogeneous Cores . . . . . 76
6.10 Load Imbalance from Skewed Content . . . . . 78
6.11 Discussion and Positioning . . . . . 80

**7 Conclusion . . . . . 83**

7.1 Threats to Validity . . . . . 84
7.2 Future Work . . . . . 85

**References . . . . . 86**

**Appendices . . . . . 89**

A Database Query Protocol . . . . . 90
9.1 Artifact Availability . . . . . 90
9.2 Database Schema . . . . . 90
9.3 Canonical Queries . . . . . 91


1

# 1 INTRODUCTION

Advancements in semiconductor technology have historically driven the growth of computational power. For decades, the number of transistors on a chip doubled roughly every eighteen months, an observation known as *Moore's Law* [1]; rising clock speeds translated these denser circuits into proportional single-core performance gains. However, in the mid-2000s, the hardware industry encountered significant physical barriers, such as excessive power consumption and challenges in heat dissipation, which limited the increase in processor clock frequency.

The response to these technological limitations was a change in processor design: instead of focusing on making a single core faster, the emphasis shifted to integrating multiple processing cores onto a single chip. Thus, multi-core processors emerged as the dominant architecture in modern computers. This transition opened up new possibilities for performance enhancement; however, to take advantage of multi-core processors, programs must be intentionally written or adapted to execute tasks in parallel [1].

This architectural shift from sequential to parallel execution has a profound impact on high-performance applications. In many scenarios, such as bioinformatics pipelines and network intrusion detection, processing large data volumes efficiently is essential. These applications rely on highly efficient multi-pattern-matching algorithms, and the Aho-Corasick algorithm stands out because it can search for many patterns simultaneously while scanning the input only once, achieving linear-time complexity with respect to the size of the input text. Widely adopted in security tools — including YARA for malware classification and Snort and Suricata for signature-based network intrusion detection [2] — the Aho-Corasick algorithm serves as a backbone for multi-pattern search engines [3].

Despite its efficiency, the Aho-Corasick algorithm is intrinsically sequential: its automaton processes input strictly left-to-right, with each state transition depending on the preceding one. As a result, when scanning a single large input on a modern multi-core CPU, the entire computational load is concentrated on one core. This serial dependency becomes a major performance bottleneck when scanning large target data, such as full disk images, packet captures, or genomic sequences. Consequently, the disparity between the algorithm's sequential nature and modern hardware's parallel

processing capabilities widens as data volumes grow [4].

## 1.1 MOTIVATION

The intrinsically sequential nature of Aho-Corasick, combined with the prevalence of multi-core architectures in general-purpose computing, presents a clear opportunity for acceleration through parallel computing. Previous studies report speedups on specialized platforms and in specific application domains [5], [2]; however, the behavior of a shared-memory, multi-threaded implementation on commodity CPUs in the *memory-bound regime* — where the automaton size far exceeds the last-level cache — has received comparatively little attention. This observation is supported by the systematic literature review presented in Chapter 3.

As pattern sets for tools such as Snort and Suricata grow in size, the corresponding Aho-Corasick automata can occupy hundreds of megabytes, pushing the hot data structure well beyond the cache hierarchy [6]. In this regime, the primary performance bottleneck shifts from computation to memory bandwidth [7], [2]. The interaction of multiple threads sharing the same last-level cache changes the scaling behavior in ways that compute-bound models do not capture. Large signature sets such as those of Snort and Suricata are one prominent example of how such automata arise in practice; understanding the limits of parallel scaling on these realistic workloads, measured on a homogeneous multi-core workstation, is the motivation for this study, with intrusion detection serving as a representative application rather than the object of the work.

## 1.2 OBJECTIVES

The general objective of this thesis is to design, implement, and evaluate a parallel version of the Aho-Corasick algorithm targeting shared-memory multi-core CPU architectures, using the POSIX Threads programming model, in order to accelerate multi-pattern matching on large inputs.

To achieve this general objective, the following specific objectives are defined:

1. To conduct a systematic literature review of parallelization and optimization techniques for the Aho-Corasick algorithm on multi-core CPUs, in order to identify the state of the art and delimit the research gap addressed by this work;

2. To design and implement a parallel version of the Aho-Corasick algorithm in C, using the pthreads API, exploiting intra-input parallelism on shared-memory multi-core CPUs;

3. To conduct a rigorous experimental evaluation on a shared-memory multi-core workstation that quantifies the performance gains of the parallel implementation in terms of speedup, throughput, and parallel efficiency, against a sequential baseline;

4. To analyze the scalability of the proposed solution under different workload, pattern-set, and thread-count configurations.

## 1.3 STRUCTURE OF THE THESIS

This document is organized as follows. Chapter 2 presents the Theoretical Framework, introducing the fundamentals of parallel computing, including architectures, performance metrics, and thread-based programming models, with emphasis on the POSIX Threads API. It also discusses the core data structures and algorithms relevant to this work, including Tries, the general pattern-matching problem, and a detailed analysis of the Aho-Corasick algorithm. Chapter 3 describes the Systematic Literature Review, presenting the research questions, search strategy, and selection criteria adopted to identify the state of the art and define the research gap. Chapter 4 presents the Proposed Solution, detailing the parallel-scanning strategy, the boundary-overlap and load-balancing mechanisms, and the flattened output layout that constitute the design. Chapter 5 describes the Methodology, specifying the workstation-based experimental environment, datasets, metrics, measurement protocol, correctness validation, and the hypotheses that the experiments are designed to test. Chapter 6 reports and discusses the Results, isolating the contribution of each optimization, including short diagnostic comparisons of the load imbalance caused by heterogeneous cores and by skewed input content, and positioning the findings against the related literature. Finally, Chapter 7 presents the Conclusion, summarizing the contributions, the threats to validity, and directions for future work.

# 2

# 2 THEORETICAL FRAMEWORK

## 2.1 FUNDAMENTALS OF PARALLELISM

Parallelism in computing refers to the simultaneous execution of multiple tasks or parts of the same task, with the primary goal of reducing a program's total execution time. It is crucial to distinguish parallelism from concurrency. Concurrency occurs when multiple tasks are in progress at a given time, but may not be executing simultaneously. A single-core system can exhibit concurrency by rapidly switching between the execution of different tasks, simulating parallelism through preemption. True parallelism, on the other hand, requires hardware with multiple processing units [8]. This distinction is fundamental to the present work, which exploits true hardware parallelism through multiple CPU cores.

### 2.1.1 Parallel Computing Architectures

Parallel computer architectures are classified according to various criteria. A prominent taxonomy, proposed by Flynn [9], categorizes these architectures based on instruction and data streams:

- **SISD (Single Instruction, Single Data)**: Represents a conventional single-processor architecture, where a single instruction operates on a single data stream;
- **SIMD (Single Instruction, Multiple Data)**: Involves multiple processing units that execute the same instruction concurrently on different data elements;
- **MISD (Multiple Instruction, Single Data)**: Characterizes systems where multiple processing units apply distinct instructions on the same data stream;
- **MIMD (Multiple Instruction, Multiple Data)**: Involves multiple processing units that execute distinct instructions on distinct data streams independently. This paradigm is directly related to multi-core architectures.

Within the MIMD category, systems are further distinguished based on their memory organization [1]:

- **Shared-Memory Systems**: In these systems, multiple processors share a physical memory address space. Inter-processor communication is typically achieved by reading from and writing to shared variables in memory. Multi-core processors are the most common and widely available example of this architecture, where all cores on a chip share access to main memory;
- **Distributed-Memory Systems**: Each processor in these systems has its own locally accessible memory, and communication between processors is usually done by exchanging messages across an interconnection network. Computer clusters are examples of this architectural model.

The present work focuses on the acceleration of algorithms in environments with multi-core processors, which fall into the shared memory MIMD category. A multi-core processor (MCP) is a single chip that contains two or more central processing units (CPUs) called 'cores'. Each of these cores is capable of reading and executing program instructions independently, essentially functioning as a complete processor.

### 2.1.2 Processes vs Threads

Within shared-memory systems, such as multi-core processors, software-level parallelism is commonly achieved through the execution of multiple threads within a single process.

A process is an instance of an executing program, an entity possessing its own dedicated virtual address space, file descriptors, environment variables, and other Operating System (OS) resources, all isolated from other processes [1].

A thread, on the other hand, is a lighter unit of execution started by a process. The same process can control, concurrently or in parallel, multiple threads. A key characteristic is that all threads within a process share its virtual address space, which encompasses executable code, global data, and the heap segment [1]. However, each thread maintains its private resources, which are essential for its independent execution context:

- A unique identifier (thread ID);
- A set of CPU registers, including the program counter (PC);
- A separate execution stack, used for local variables and function call control;
- Thread-specific state, such as signal masking and scheduling priority.

### 2.1.3 POSIX Thread Model (Pthreads)

The POSIX Thread Model, commonly referred to as "Pthreads", is a standard that defines an application programming interface (API) for thread creation and management, as specified by IEEE Std 1003.1c-1995. Pthreads provides a portable suite of

data types, functions, and constants that allows developers to create multithreaded applications that are compilable and executable across diverse POSIX-compliant operating systems (e.g., Linux, macOS, and various Unix variants). The Pthreads API provides functionality for managing the thread lifecycle—including creation, termination, and joining—and, critically, incorporates synchronization primitives. These primitives, such as mutexes and condition variables, are essential for managing access to shared resources within shared-memory environments [8] [1].

### 2.1.4 Cache Architecture and Memory-Bound Parallelism

Modern shared-memory processors organize their caches in a hierarchy. Each core has private L1 and L2 caches—typically tens to hundreds of kilobytes—for low-latency access to recently used data. A shared last-level cache (LLC), often several megabytes, is accessible to all cores on the chip and serves as the final caching tier before main memory [1].

A workload is *compute-bound* when its execution time is dominated by arithmetic operations, leaving memory bandwidth underutilized. It is *memory-bandwidth-bound* (or simply *memory-bound*) when the bottleneck is the rate at which data can be fetched from DRAM rather than computational throughput. Pattern-matching algorithms that operate on large automata—whose transition tables exceed the LLC capacity—fall into this regime: each state transition triggers a cache miss that must be resolved from DRAM, making throughput a function of available memory bandwidth rather than CPU frequency or core count [10].

Parallelism can intensify memory pressure. When multiple threads simultaneously traverse different regions of a large, read-only automaton, the combined working set exceeds the LLC, raising the aggregate cache-miss rate for all threads. Two additional hazards arise when threads share writable data. *Cache coherence* protocols (such as MESI) ensure that modifications made by one core become visible to all others; they introduce inter-core invalidation traffic whenever a shared cache line is written [1]. *False sharing* occurs when two threads write to logically independent variables that happen to reside on the same cache line, causing coherence invalidations despite the absence of a true data dependency. False sharing can be avoided by aligning per-thread accumulators to cache-line boundaries.

Non-Uniform Memory Access (NUMA) architectures differentiate memory latency based on which socket owns the physical memory being accessed. The experimental platform used for the main measurements in this work is a single-socket Uniform Memory Access (UMA) machine; nevertheless, NUMA is an important consideration in multi-socket server deployments where memory-bound workloads exhibit significant latency asymmetry [10].

### 2.1.5 Hybrid P/E-Core Architectures

Modern consumer and mobile processors have introduced a *hybrid* microarchitecture that integrates two distinct core types on a single die. A typical Alder Lake mobile processor pairs high-performance cores (P-cores), often with simultaneous multithreading, with energy-efficient cores (E-cores) that prioritize throughput per watt.

P-cores feature wider out-of-order execution pipelines and higher operating frequencies, delivering greater per-thread throughput at higher power consumption. E-cores are physically smaller and optimized for throughput-per-watt rather than single-thread latency; they operate at lower frequencies. When equal-size work units are assigned across both core types, the faster cores can finish early and wait at a synchronization barrier while slower cores complete their share. This is the architectural reason for the frequency-weighted and dynamic load-balancing strategies introduced later: they are mechanisms for reducing stragglers on heterogeneous processors, not a change to the Aho–Corasick matching semantics. The main experimental campaign of this thesis is conducted on homogeneous cores; the hybrid-core discussion appears only as a diagnostic comparison in the results chapter.

## 2.2 PARALLEL PERFORMANCE EVALUATION METRICS

Performance evaluation is fundamental to understanding the effectiveness of a parallel program. The following metrics compare the execution time of the parallel program with its equivalent sequential counterpart [10]. The evaluation of parallel algorithms requires objective metrics that quantify both performance gains and resource utilization; accordingly, the metrics presented in this section provide the basis for evaluating the implementation proposed in this work and for interpreting the experimental results reported in Chapters 5 and 6.

### 2.2.1 Execution Time and Overhead

The performance of a program is measured by its execution time. For parallel programs, the following terms are defined:

- **Ts**: The execution time of a baseline sequential algorithm executed on a single processor unit.
- **Tp**: The execution time of the parallel program utilizing p processors.

Ideally, `Tp` is expected to be substantially less than `Ts`. However, parallel execution introduces *overhead*. This encompasses the time spent by processors on tasks that do not directly contribute to problem resolution, including communication, synchronization, load balance, and other computations inherent to the parallel implementation [10].

### 2.2.2 Speedup

Speedup (S) measures the performance gain achieved through parallel execution relative to sequential execution, as shown in Eq. 1. It is defined as the ratio of sequential execution time to parallel execution time:

$$S = \frac{T_s}{T_p} \tag{1}$$

An ideal, or linear, speedup would be `S = p`. Amdahl's Law [11] establishes a theoretical upper bound on the speedup achievable by parallelizing a program with a fixed problem size. If `s` represents the fraction of a program's sequential execution time that is inherently serial (non-parallelizable), and `f` is the parallelizable fraction (`s + f = 1`), the maximum speedup on `p` processors is given by Eq. 2.

$$S \leq \frac{1}{s + f/p} \tag{2}$$

In the limit, as the number of processors (p) tends to infinity, the maximum speedup is bounded by $\frac{1}{s}$. This implies that even a small sequential fraction (small s) can severely limit the total speedup, regardless of the number of processors available. Figure 1 illustrates the theoretical speedup according to Amdahl's Law compared to ideal speedup for a given sequential fraction.

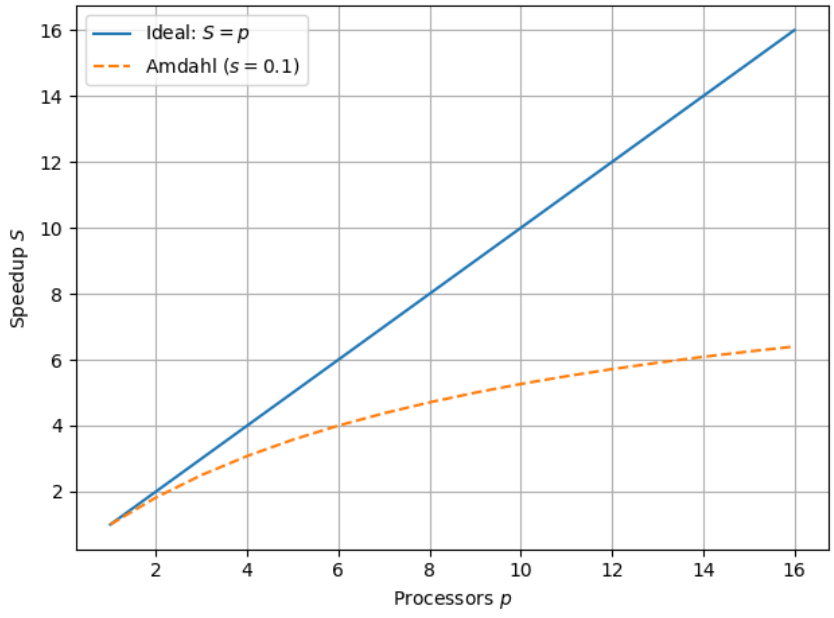



Figure 1: Ideal vs. Amdahl's theoretical speedup ($s = 0.1$).

A complementary perspective is offered by the Gustafson-Barsis Law [12] in Eq. 3, where $\alpha$ denotes the fraction of the *parallel* run time spent in non-parallelizable (serial) work (and is therefore a distinct quantity from Amdahl's $s$, which is the serial fraction

of the *sequential* run). This law argues that, for problems whose size can be scaled with the available processors, an approximately linear speedup is attainable even in the presence of a serial fraction. Both Amdahl's and Gustafson's laws are important for understanding the limits and potential of parallelism in different scenarios [1].

$$S_{\text{scaled}} = \alpha + (1 - \alpha) \cdot p \tag{3}$$

### 2.2.3 Efficiency

Efficiency (E) measures the effectiveness of processor utilization in a parallel program. It is defined as the ratio of speedup to the number of processors p, as in Eq. 4:

$$E = \frac{S}{p} = \frac{T_s}{p \cdot T_p} \tag{4}$$

Efficiency values range between 0 and 1 (inclusive), often expressed as a percentage (0% to 100%).

### 2.2.4 Scalability

In parallel computing, scalability denotes a system's ability to sustain high performance (often via efficiency) as both the number of processors and/or the problem size grow [10].

- **Strong Scalability**: Measures how execution time changes when the processor count $p$ increases for a fixed total problem size, denoted $W$ (for example, the total number of array elements to process). The goal is to keep efficiency nearly constant as $p$ increases. Amdahl's Law directly applies in this scenario;
- **Weak Scalability**: Measures how execution time changes when both the processor count $p$ and the total problem size $W$ increase such that the work per processor ($W/p$) stays constant. The aim is to maintain efficiency as $p$ and $W$ grow proportionally. Gustafson's Law is most relevant here.

A parallel system is considered scalable if its efficiency remains above an acceptable threshold as both the number of processors $p$ and the problem size $W$ increase.

## 2.3 TRIES (PREFIX TREES)

This section explains the Trie data structure, detailing its defining properties, core operational mechanics, performance aspects, and its significance as a foundational element in advanced string processing algorithms, particularly the Aho-Corasick algorithm.

### 2.3.1 Definition and Basic Structure

A Trie—also known as a prefix tree—is a specialized tree-like data structure optimized for the efficient storage and retrieval of a dynamic set of strings. Edward Fredkin introduced the term “trie” in 1960, highlighting its primary application in information retrieval [13], [14]. Figure 2 illustrates an example of a trie storing a set of words.

Tries store strings by organizing nodes so that each path from the root to a node represents a unique prefix. In a standard $R$-way trie, each node can branch to $R$ children, where $R$ denotes the size of the character alphabet. The existence of a parent-child edge highlights two key aspects of tries:

- The edge’s position (index) within the parent node’s array of $R$ possible child links directly corresponds to a unique character in the alphabet. For instance, index 0 might correspond to ‘a’, index 1 to ‘b’, and so on, according to a predefined mapping.
- The character associated with this edge extends the prefix represented by the parent; this implies that at least one word (or key) in the trie contains this character sequence.

To signify that such a prefix also constitutes a complete key present in the trie, the node reached at the end of this path is typically marked by storing an associated non-null value or a boolean flag. The characters themselves are implicitly encoded by the trie’s structure rather than being explicitly stored within each node [15].

### 2.3.2 Core Operations

Tries support several primary operations, including search, insertion, and deletion [15]. These operations are explained below:

- **Search**: locating a key involves traversing the trie starting from the root, following the successive characters of the target key. Each character dictates which child link to follow. The search operation succeeds if a complete path corresponding to all characters of the key exists and the node reached at the end of this path is appropriately marked as representing a complete key. The search fails if, at any point, a required link is null (i.e., no path for the current character exists) or if the path is fully traversed but the final node is not marked as a complete key;
- **Insertion**: the insertion process begins similarly to a search, by traversing the trie according to the characters of the key to be added. If the path for the key does not extend to its full length (i.e., a null link is encountered), new nodes are created and linked to form the remainder of the path. The node corresponding to the final character of the inserted key is then marked to indicate that it is a complete key in the set;

- **Deletion**: deleting a key first involves locating the node corresponding to the key's final character and removing its end-of-word marker. If there are no child links (meaning it is not a prefix for any other key), it can be deleted. This deletion may proceed recursively: when a child node is removed, if its parent has no other children and the parent node is not marking a key, the parent node becomes redundant and can be deleted. This process continues up to the root as long as parent nodes become redundant.

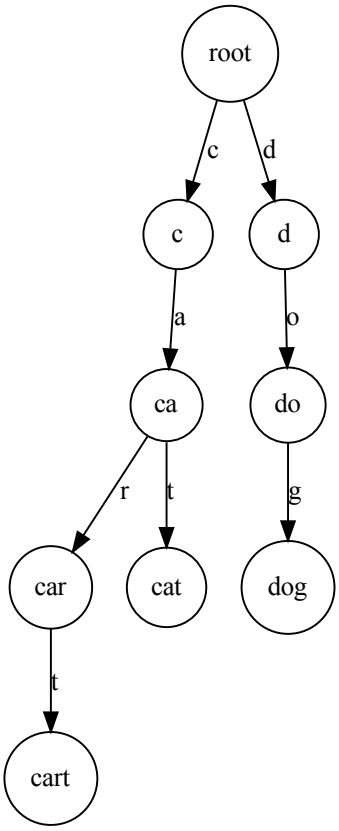



Figure 2: Example trie storing {"car", "cat", "cart", "dog"}. Nodes marking the end of a complete word are shown with a double circle.

### 2.3.3 Advantages and Performance Characteristics

Tries provide distinct advantages for operations on string collections, as detailed below.

- **Time Complexity**: a primary advantage is that the time required for search and insertion operations depends primarily on the length of the key ($L$), not on the total number ($N$) of keys stored. Specifically, these operations typically require

examining a number of nodes proportional to $L$. Sedgewick and Wayne [15] state that the number of array accesses (node visits) is at most $L+1$. This $O(L)$ performance makes tries highly efficient for prefix-based searches (e.g., retrieving all keys starting with a given prefix) and ensures that search performance does not substantially degrade as the dataset size increases;

- **Prefix Sharing**: when stored strings share common prefixes, tries inherently exploit this by having those strings share the initial path from the root. This sharing can lead to significant space savings compared to storing each string independently, particularly if there is substantial prefix overlap among keys [13].

While prefix sharing can save space, the overall space complexity of $R$-way tries requires careful consideration. Each node in an $R$-way trie conceptually provides $R$ potential links. If the alphabet size $R$ is large (e.g., for Unicode) and the actual branching factor at most nodes is low (a sparse trie), allocating space for all $R$ links per node can lead to considerable memory consumption due to numerous unused (null) links [14], [15].

In summary, the trie data structure is a cornerstone for numerous advanced string processing algorithms. Critically for this work, the Aho-Corasick algorithm, engineered for the efficient, simultaneous matching of multiple patterns within a text, employs a trie—constructed from the set of patterns (the dictionary)—as its initial structural framework [4]. A thorough grasp of the trie principles presented above is thus essential for developing and analyzing the Aho-Corasick algorithm.

## 2.4 PATTERN MATCHING

This section introduces the fundamental problem of pattern matching within strings, explores key algorithmic approaches to its solution, and highlights its significance across various computational domains. This foundational discussion provides the essential context for understanding advanced multi-pattern matching algorithms, such as the Aho-Corasick algorithm. In general information processing, pattern (or *string*) matching underpins the functionality of text editors and web search engines for keyword searches. The field of computational biology extensively utilizes pattern matching for analyzing genetic sequences [15]. Furthermore, in the field of information security, pattern matching is crucial: Network Intrusion Detection Systems (NIDS) utilize it to identify malicious signatures in network traffic. Tools like YARA, for instance, define rules based on textual or binary patterns to classify and identify malware [3].

### 2.4.1 The Pattern Matching Problem

The *string matching* problem consists of determining whether a string $P$ (the *pattern*) of length $M$ occurs within a string $T$ (the *text*) of length $N$, where $M \leq N$. Formally, it

seeks all positions $pos$ such that the condition in Eq. 5 holds for every $i \in \{0, 1, \ldots, M-1\}$, and each valid $pos$ corresponds to an occurrence of the pattern in the text:

$$P[i] = T[pos + i], \quad \text{with} \quad pos + i < N \tag{5}$$

Beyond simply finding the first match, the problem often extends to locating all occurrences, enumerating them, or extracting surrounding contextual information for each match [15].

Algorithms that address the string matching problem are generally categorized based on the number of patterns they process and the exactness of the match. This thesis focuses on *Exact Matching*, where the pattern must correspond precisely to a segment of the text. A key distinction for this study lies between algorithms designed for *Single-Pattern Matching* and those optimized for *Multi-Pattern Matching*, where the objective is to identify all occurrences of any pattern from a predetermined set of patterns within a given text [4], [16].

### 2.4.2 The Naive Algorithm

The naive or brute-force algorithm represents the most straightforward method for exact pattern matching. Its operation involves iteratively aligning the pattern $P$ with the text $T$ at every possible starting position $i$ (from 0 to $N - M$). For each alignment, $P$ is compared character by character, from left to right, against the corresponding substring. If all characters match, an occurrence is reported [16].

**Algorithm 1** Naive exact string matching

**Require:** text $T[0 \ldots N-1]$, pattern $P[0 \ldots M-1]$
**Ensure:** all starting positions where $P$ occurs in $T$

```
for i ← 0 to N − M do
    j ← 0
    while j < M and T[i + j] = P[j] do
        j ← j + 1
    end while
    if j = M then
        report occurrence at position i
    end if
end for
```

Following this comparison (whether a match or a mismatch), the algorithm shifts $P$ one position to the right relative to $T$ and proceeds to the next alignment. Despite its simplicity, Knuth, Morris, and Pratt (1977) state that this method can be highly inefficient [17]. Its worst-case time complexity is $O(M \cdot N)$ character comparisons. This

occurs, for example, when searching for a pattern like $P = a^n b$ within a text $T = a^{2n} b$, where many partial matches occur. To address these limitations, more advanced pattern matching algorithms include a preprocessing phase for the pattern $P$. This phase extracts structural information about $P$ that is then utilized during the search, enabling larger shifts of the pattern relative to the text and thus reducing the total number of character comparisons.

### 2.4.3 Knuth-Morris-Pratt (KMP) Algorithm

The KMP algorithm [17] efficiently resolves the pattern matching problem with an optimal worst-case time complexity of $O(N + M)$. Unlike the naïve approach, KMP avoids re-checking characters that have already been matched. To achieve this, KMP first preprocesses the pattern $P$ by constructing a special array known as the *failure array* (or *prefix array*), typically denoted by $p[1..M]$. This array indicates the length of the longest prefix that is also a suffix for each prefix of $P$. With this information, when a mismatch occurs during the search, the algorithm uses the failure array to decide which pattern index to resume comparison from, so the text pointer never backtracks, bypassing unnecessary comparisons and preventing redundant checks.

### 2.4.4 Boyer-Moore (BM) Algorithm

The Boyer-Moore (BM) algorithm [18] is widely recognized for its excellent practical performance, often outperforming linear-time methods in typical cases. Unlike the KMP algorithm, Boyer-Moore typically compares characters starting from the end of the pattern and moves backward. Its efficiency primarily stems from two rules (or heuristics):

- **Bad Character Rule**: When a mismatch occurs, this rule uses the character from the text that caused the mismatch to determine how far the pattern can safely shift. The algorithm shifts the pattern to align this mismatched text character with its last occurrence in the pattern or moves completely past the mismatched character if it does not appear in the pattern;
- **Good Suffix Rule**: When a mismatch happens after matching one or more characters from the end of the pattern, this rule uses the matched suffix to decide how far to shift the pattern. It searches for another occurrence of the matched suffix within the pattern, ensuring that no previously matched characters are unnecessarily compared again.

These two rules combined enable the Boyer-Moore algorithm to skip larger portions of the text, significantly reducing the total number of comparisons.

### 2.4.5 The Need for Specialized Multi-Pattern Algorithms

When the task involves searching for multiple patterns simultaneously from a set $K = \{P_1, P_2, \ldots, P_k\}$ within a text $T$, simply applying a single-pattern matching algorithm for each $P_i$ independently proves highly inefficient. This "iterative" approach requires scanning the text multiple times, resulting in a total time complexity that can be approximately $O(k \cdot (N + M_{\text{avg}}))$ or worse, where $M_{\text{avg}}$ is the average pattern length [4].

To efficiently handle applications that require simultaneous searches for a large number of patterns, specialized algorithms are essential. These methods usually preprocess the entire set of patterns to create a unified data structure, which enables a single, efficient pass over the text to identify all occurrences of any pattern within the set [16].

The theoretical framework of *finite automata* provides a robust and elegant solution to this problem. By constructing a single *finite automaton* (FA) that can recognize all patterns in a given set, the text can be processed in a single continuous scan. Each character read from the text triggers a *state transition* within the FA. When the FA reaches an *accepting state*—a state explicitly designed to indicate the successful recognition of one or more patterns—the algorithm reports an occurrence. Algorithms like the Aho-Corasick algorithm exemplify this approach; they automate the construction of such a *pattern-matching machine*, typically based on a trie structure augmented with specialized *failure transitions* [4]. This automaton-based paradigm forms the critical foundation for efficiently solving multi-pattern matching problems. The Aho-Corasick algorithm, which instantiates this paradigm and is the focus of this work, is examined in detail in the next section.

## 2.5 THE AHO-CORASICK ALGORITHM

This section provides a comprehensive examination of the Aho-Corasick algorithm, a highly efficient string-matching method designed to locate all occurrences of multiple patterns (keywords) within a given text. It combines *automata theory* with *trie* data structures to efficiently solve the *multi-pattern search* problem. Its search phase runs in $O(n + z)$ time, where $n$ is the text length and $z$ is the number of matches [4].

### 2.5.1 Context

The task of concurrently identifying all instances of a large collection of patterns within a text poses a significant computational challenge. A naïve approach—running a *single-pattern matching* algorithm such as KMP or BM for each of the $k$ patterns—requires repeated scans of the text and has a worst-case time complexity of $O(k \cdot n)$, where $k$ is the number of patterns and $n$ is the text length, rendering it impractical for large pattern sets or long texts [16].

The Aho-Corasick algorithm overcomes this by building one *finite automaton* that

encodes all patterns simultaneously. This automaton processes the input text in a single pass to detect all matches. The algorithm effectively combines the *state-transition* efficiency of finite automata with the prefix-sharing advantages of tries, following *failure links* on mismatches and resuming matching without rescanning characters—an extension of KMP's *failure-function* idea to a trie of patterns [16].

### 2.5.2 Algorithm Components and Construction

Aho-Corasick has two main phases: (i) construction, which builds the automaton, and (ii) search, which scans the text. The automaton has three key tables, built in the following order:

- **Keyword trie (`goto`):** a *keyword trie* is first constructed from the pattern set $K = \{P_1, P_2, \ldots, P_k\}$. Each node represents a state, and each edge corresponds to a character transition. A path from the root to any state $s$ spells a prefix of one or more keywords. Formally, the transition function $\texttt{goto}(s, a)$ returns the next state $s'$ when an edge labelled $a$ exists; otherwise the transition fails. Undefined transitions from the root are assigned back to the root, i.e., $\texttt{goto}(0, a) = 0$, creating implicit self-loops for every alphabet symbol;

- **Failure function (`fail`):** for any state $s$ (except the root), $\texttt{fail}(s)$ is the state $s'$ whose path label is the longest proper suffix of the path label of $s$ that is also a prefix of some keyword. On a mismatch, the automaton follows $\texttt{fail}(s)$ and resumes matching without consuming the current character. The failure function `fail` is computed level-by-level (breadth-first) after the `goto` table is complete;

- **Output function (`output`):** the output function $\texttt{output}(s)$ identifies all keywords that end at state $s$. For a given state $s$, $\texttt{output}(s)$ is the set of all patterns $P_i$ in $K$ such that the string corresponding to the path to $s$ is $P_i$. To ensure all matches are reported, including those that are suffixes of other patterns, $\texttt{output}(s)$ is augmented during the computation of `fail`: if $\texttt{fail}(s) = s'$, then $\texttt{output}(s) \leftarrow \texttt{output}(s) \cup \texttt{output}(\texttt{fail}(s))$. This incremental union lets each state report every pattern that ends either at that state or at any suffix state reached through its failure chain.

The complete automaton, combining the `goto` transitions, `fail` links, and `output` match sets, is illustrated in Figure 3 for the pattern set $K = \{\text{“he”}, \text{“she”}, \text{“hers”}\}$.

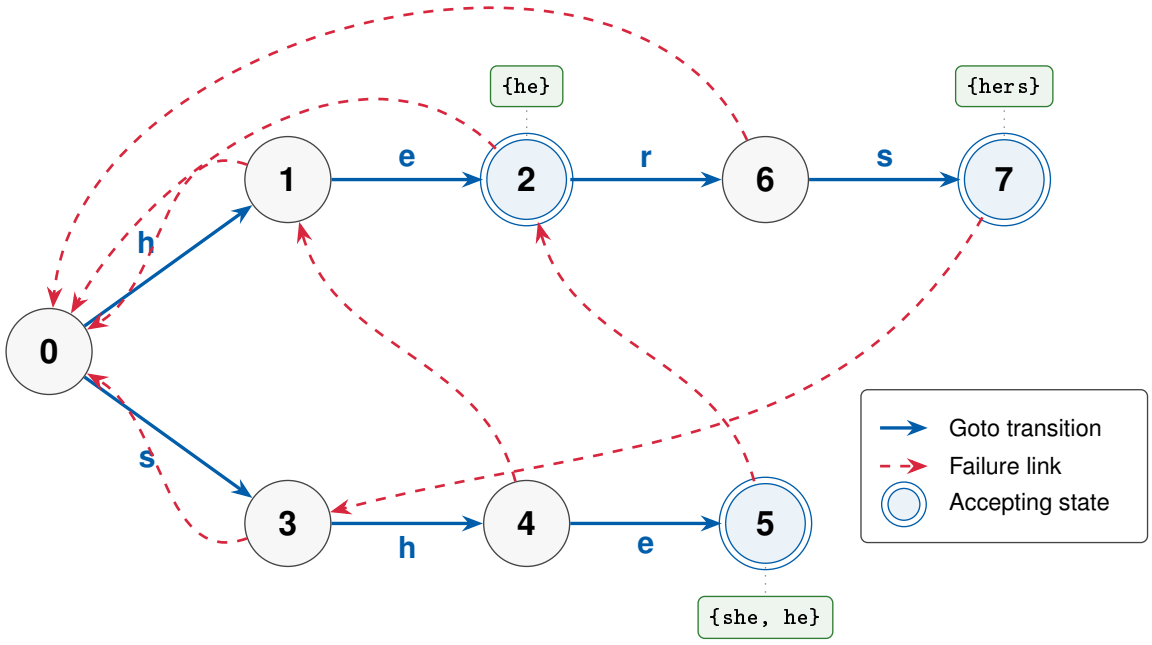



Figure 3: Complete Aho–Corasick automaton for the pattern set $K = \{\text{“he”}, \text{“she”}, \text{“hers”}\}$, showing solid blue `goto` transitions, dashed red `fail` links, double-circled accepting states, and their corresponding `output` match sets.

To make the interaction of the three tables concrete, consider scanning the text `ushers` with the automaton of Figure 3. The leading `u` has no outgoing edge from the root, so the implicit self-loop keeps the machine at state $0$. The substring `she` then drives three successive `goto` transitions to the accepting state for `she`; because the failure link of that state points to the state for `he`—the longest proper suffix of `she` that is also a keyword prefix—its augmented `output` set reports both `she` and `he` at that position. The next character `r` has no `goto` edge from the `she` state, so the automaton follows the failure link to the `he` state and continues along `her` and then `hers`, emitting `hers` when the final `s` is consumed. A single left-to-right pass thus reports every occurrence—`he`, `she`, and `hers`—without ever rescanning a character.

### 2.5.3 Matching Phase

Once the Aho–Corasick automaton has been built, it scans the text $T = a_1 a_2 \dots a_n$ in one left-to-right pass. At each position $i$, the machine attempts to follow the transition labelled $a_i$ from its *current_state* ($cs$); if that edge exists, it moves along it, otherwise it follows *failure links* (`fail`) until either a matching transition is found or the root is reached. Every time a successful transition is made, the automaton inspects the `output`

set of the new state and emits all patterns ending at position $i$.

Figure 4 illustrates this control flow in detail, which: starts at the root with $i = 1$, attempt each $\texttt{goto}(cs, a_i)$, follow failure links on a “no” decision, report matches via $\texttt{output}(cs)$ on a “yes,” advance $i$, and terminate when $i > n$.

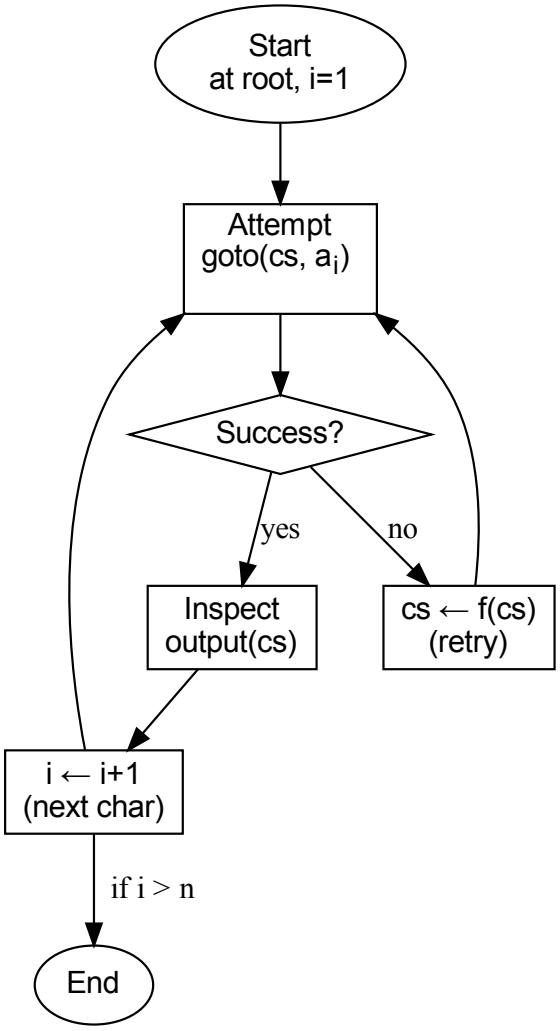



Figure 4: Flowchart of the Aho–Corasick matching phase.

Each input character advances the automaton by at most one $\texttt{goto}$ transition, which increases the current depth by one, whereas every failure-link traversal strictly decreases that depth. Since the depth is non-negative, the total number of failure transitions over the scan cannot exceed the number of $\texttt{goto}$ transitions, which bounds the total number of transitions ($\texttt{goto}$ plus $\texttt{fail}$) by $< 2n$ [4].

### 2.5.4 Time and Space Complexity

The Aho-Corasick algorithm exhibits optimal *linear time complexity* for multi-pattern matching [4], [16]. The complexities for the construction and search phases and storing the automaton are as follows:

- **Construction phase**: building the `goto`, `fail`, and `output` functions takes $O(L)$ time, where $L$ is the total length of all keywords. If a dense array is used for the root, an additional $O(|\Sigma|)$ initialization cost is incurred, giving $O(L + |\Sigma|)$;
- **Search phase**: Scanning a text of length $n$ performs fewer than $2n$ state transitions. Reporting matches costs time proportional to the total number of occurrences $z$, so the total search time is $O(n + z)$;
- **Space complexity**: Storing the automaton requires $O(L + |\Sigma|)$ space: $O(L)$ for trie edges, $O(L)$ pointers for failure links, and additional space for output lists (proportional to the number of stored outputs).

### 2.5.5 NFA versus DFA Representations

The `goto`-plus-`fail` automaton described above is commonly distinguished, in the network-intrusion-detection literature, as the *nondeterministic finite automaton* (NFA) configuration of Aho–Corasick, in contrast to the fully precomputed *deterministic finite automaton* (DFA) discussed below [19]. This NFA label is conventional rather than formal—the `goto`-plus-`fail` machine in fact behaves deterministically—but it captures the operational difference: the `goto` function is undefined on many (state, character) pairs, and the search algorithm resolves those gaps by traversing failure links at runtime, so a single character may require several memory accesses in the worst case (the cost is constant only in the amortized sense established above). A widely adopted optimization converts this NFA into a *deterministic finite automaton* (DFA) by precomputing the failure-link chains during construction: for every state $s$ and character $a$, the full deterministic transition $\delta(s, a)$ is stored in a flat two-dimensional table of size $|Q| \times |\Sigma|$, where $|Q|$ is the number of states and $|\Sigma|$ is the alphabet size [4], [16]. With this representation every character in the input triggers exactly one array lookup; no runtime failure-link traversal occurs, which reduces per-character latency and simplifies the inner search loop.

The cost of the DFA representation is memory: for the standard 256-symbol byte alphabet the flat transition table occupies $4 \cdot |Q| \cdot 256$ bytes. This footprint grows linearly with the pattern-set size and can easily exceed the processor's last-level cache (LLC). When the table no longer fits in cache, each state transition becomes a potential cache miss, shifting the workload from compute-bound to *memory-bandwidth-bound*.

The implementation in this work uses the DFA (flat transition table) representation exclusively; the `goto`, `fail`, and `output` functions described in the preceding subsection are intermediate construction artefacts that are not retained at search time.

### 2.5.6 Applications and Relevance

The Aho-Corasick algorithm is widely used in applications that require high-performance *multi-pattern matching*, where its single-pass processing of the input against the entire pattern set is essential. Representative use cases include large-scale text analysis, motif searching in DNA or protein sequences [5], spam filtering, Network Intrusion Detection Systems (NIDS) that scan traffic against thousands of signatures, and anti-malware tooling—YARA, for example, employs Aho-Corasick to match the literal string components extracted from its rules [3].

While Aho-Corasick excels at exact literal matching, systems like NIDS often layer additional capabilities—such as *regular-expression matching*—on top of Aho-Corasick engines to support more expressive signatures, which affects overall performance characteristics. The algorithm therefore remains a theoretically elegant and practically impactful example of *finite-automata theory* applied to string processing [16], [3].


3

# 3 SYSTEMATIC REVIEW

This section outlines the methodology employed for the Systematic Literature Review (SLR) conducted to identify, evaluate, and synthesize pertinent studies concerning the parallelization and optimization of the Aho-Corasick algorithm. The review focuses on implementations targeting multi-core CPUs. The objective of this SLR is to support the development of the present research and to answer the formulated research questions.

## 3.1 RESEARCH QUESTIONS

The following Research Questions (RQs) were formulated to guide the literature search, selection, and analysis process:

1. **RQ1:** What parallelization techniques and associated challenges are documented in the literature for implementing the Aho-Corasick algorithm utilizing threads on multi-core CPUs?

2. **RQ2:** What specific optimizations for the Aho-Corasick algorithm are employed in application scenarios involving extensive pattern sets, particularly within domains of information security?

These research questions were formulated based on the objectives of this work and aim to identify both parallelization strategies and optimization techniques applicable to the proposed implementation.

## 3.2 SEARCH STRATEGY AND STUDY SELECTION PROTOCOL

The search and selection protocol for identifying primary studies encompassed the definition of data sources, construction of search queries, and establishment of inclusion and exclusion criteria.

### 3.2.1 Data Sources

The following scientific databases were selected due to their scope and relevance in the area of Computer Science and Software Engineering:

- Web of Science (WoS)
- Scopus

- ACM Digital Library (ACM DL)
- IEEE Xplore

### 3.2.2 Search Queries

Two primary search strings were constructed to address the aspects of parallelization (RQ1) and optimization/application (RQ2) pertinent to the Aho-Corasick algorithm. The literature search was executed in May 2025.

**String 1 (Focus on Parallelization):**

```
"Aho-Corasick" AND (parallel* OR multithread* OR multi-thread* OR thread* OR concurren* OR hpc OR high-perform* OR high perform*) AND ("multi-core" OR multicore OR "many-core" OR CPU) NOT "distributed systems"
```

**String 2 (Focus on Optimization and Applications):**

```
"Aho-Corasick" AND (optimi* OR accelerat* OR perform* OR evaluat* OR improve*) AND ("application" OR "implementation" OR "use case")
```

### 3.2.3 Inclusion (IC) and Exclusion (EC) Criteria

The subsequent criteria were established for the selection of primary studies:

**Inclusion Criteria (IC):**

- **IC1 (Primary Focus):** The study addresses the Aho-Corasick (AC) algorithm or presents parallelization/optimization techniques demonstrably applicable to AC.
- **IC2 (Parallelism Aspect):** The study investigates, proposes, implements, analyzes, or explicitly discusses the parallelization or concurrent execution of the Aho-Corasick algorithm.
- **IC3 (Target Platform):** The study reports empirical results or analyses about multi-core CPU platforms.
- **IC4 (RQ Relevance):** The study provides information relevant to addressing at least one RQ.
- **IC5 (Language):** Published in English or Portuguese.
- **IC6 (Accessibility):** Full-text availability of the study.
- **IC7 (Publication Window):** Published between January 2015 and May 2025, inclusive.

**Exclusion Criteria (EC):**

- **EC1 (Irrelevant Focus):** Studies with only superficial mentions of the Aho-Corasick algorithm or pattern matching, lacking a primary focus on its parallelization or optimization.
- **EC2 (Non-CPU Platform Exclusivity):** Studies exclusively focused on non-CPU platforms (e.g., GPU, FPGA) without analysis or insights transferable to multi-core CPU environments.
- **EC3 (Insufficient Detail):** Studies lacking sufficient detail regarding methodology, implementation, or reported results.
- **EC4 (Publication Type):** Opinion pieces, editorials, and non-peer-reviewed gray literature.
- **EC5 (Language Barrier):** Publications in languages other than English or Portuguese.
- **EC6 (Duplication):** Duplicate publications (the most comprehensive or recent version was retained).
- **EC7 (Inaccessibility):** Full-text unobtainable.

### 3.2.4 Selection Process and Initial Search Outcomes

The study selection was a multi-stage process. Initially, the constructed search strings were executed against the selected digital libraries. The retrieved results were subsequently refined using filters provided by each database (e.g., publication period, document type, language, subject area), as detailed in Table 1:

Table 1: Search results and filters applied per database.

| **Database** | **String 1** (Found / Kept) | **String 2** (Found / Kept) | **Filters Applied** |
|---|---|---|---|
| ACM Digital Library | 122 / 29 | 274 / 66 | 2015-2025; Research Article; Publisher: ACM; Title excl.: GPU; Lang: English. |
| Web of Science | 30 / 6 | 66 / 20 | 2015-2025; WoS Cats: CS (excl. AI); Title excl.: GPU; Lang: English. |
| Scopus | 34 / 6 | 145 / 49 | 2015-2025; Lang: English; Subj: CS, Eng; Types: Article, Conf. Paper; Title excl.: GPU. |
| IEEE Xplore | 25 / 2 | 60 / 18 | 2015-2025; Title excl.: GPU; Lang: English. |

The `Title excl.: GPU` filter listed in Table 1 acted only as a coarse identification-stage noise reducer, not as the formal platform criterion: the platform decision is governed by EC2, under which studies on non-CPU platforms (GPU, FPGA, ASIC, SIMD) were retained whenever they offered techniques or findings transferable to shared-memory multi-core CPUs, as argued individually in each review below.

Following the initial database search and filtering stage, the resultant sets of candidate articles were aggregated. The 196 records retained across both search strings and all four databases (Table 1) were merged; after removing duplicate entries and screening the titles and abstracts of the remainder against the inclusion and exclusion criteria, 24 unique studies were pre-selected. From these 24, the 10 most directly pertinent to the research questions were judged most relevant and were summarized as the primary studies for the detailed review that follows.

## 3.3 REVIEW OF SELECTED STUDIES

The selection process described above yielded the corpus of primary studies analyzed in this review. The following subsections examine in detail the ten selected primary studies, spanning shared-memory multi-core parallelizations of Aho–Corasick (RQ1) and optimizations of the algorithm in security-oriented matching workloads (RQ2); each review summarizes the problem addressed, the proposed technique, the experimental evidence, and its relation to the present design.

### 3.3.1 A Parallel Algorithm of String Matching Based on Message Passing Interface for Multicore Processors

Qu et al. [19] address the limitations of classical multiple-pattern string matching algorithms, such as the Aho-Corasick algorithm, which were originally designed for single-core systems and often become bottlenecks in modern network intrusion detection systems under heavy traffic. The authors highlight the need to use the parallel computation power of modern multicore processors to maintain high performance in deep packet inspection.

To solve this performance bottleneck, the researchers propose a parallel Aho-Corasick string matching algorithm that runs on a multicore platform but coordinates its workers through the Message Passing Interface (MPI), a distributed-memory programming model applied here within a single shared-memory node. The core methodology relies on data parallelism: a master thread constructs the automaton sequentially and partitions the input text into contiguous blocks. To prevent missed cross-boundary matches, these blocks overlap by the length of the longest pattern minus one character. The data chunks are then distributed to multiple slave threads via MPI communication, allowing each worker to independently traverse the read-only automaton.

The proposed solution was evaluated on an 8-core multicore system using Snort IDS rulesets of 10,000 and 30,000 patterns. The results demonstrate that the Deterministic Finite Automaton (DFA) variant achieves a peak average throughput of 10.5 Gbps, outperforming the Nondeterministic Finite Automaton (NFA) approach in speed and stability, albeit with higher memory consumption. Furthermore, the evaluation shows that throughput scales monotonically with the addition of CPU cores and thread-level parallelism.

Ultimately, the paper demonstrates that a master-slave data partitioning strategy with overlapping boundaries is highly effective for parallelizing the Aho-Corasick algorithm on multicore processors. However, the reliance on MPI for a single shared-memory node introduces serialized communication overhead, which highlights the advantages of using native shared-memory threading models, such as POSIX Threads, for lock-free coordination.

### 3.3.2 A Flexible Pattern-Matching Algorithm for Network Intrusion Detection Systems Using Multi-Core Processors

Lee and Yang [2] tackle the throughput bottleneck imposed by Aho-Corasick-based pattern matching in signature-based Network Intrusion Detection Systems (NIDS), where the matching phase can account for up to 70%

To overcome this rigidity, the researchers propose the Flexible Head-Body Matching (FHBM) algorithm. Instead of partitioning the automaton at a fixed depth, FHBM employs a greedy procedure that fills the head with up to a configurable maximum number of states (`HSIZE`), prioritizing parent nodes with the largest number of children and preserving the invariant that all siblings remain on the same side of the partition. The head is represented as a full 256-entry transition table for $O(1)$ lookup, while the body is compacted into an NFA traversed via 128-bit SIMD comparisons. Parallel execution is realized at the thread level: four worker threads, one per physical core, concurrently traverse the read-only Head-Body Finite Automaton (HBFA) on the input stream, exploiting the immutability of the automaton after construction to avoid synchronization in the search phase.

The evaluation was conducted on a quad-core Intel Core i7-3770 (3.4 GHz, 8 MB L3) running 64-bit Ubuntu 14.04, using pattern dictionaries extracted from the Snort ruleset (8,673 patterns) and the ClamAV signature database (5,248 and 2,899 patterns for the type-1 and type-3 sets, respectively). Synthetic input streams were generated by embedding pattern prefixes into a clean corpus – the King James Bible (HTML) for Snort and the concatenation of `/usr/bin` binaries for ClamAV – with controlled match ratios of 1%

Overall, the paper demonstrates that a flexible, state-count-based partitioning of the AC-DFA, combined with SIMD body evaluation and multi-core data parallelism,

achieves good throughput on commodity hardware. It also identifies cache capacity as the dominant performance ceiling: once the active state memory exceeds the L3 budget, additional states no longer improve throughput, a finding that reinforces the cache-locality concerns inherent in any shared-memory parallel implementation of Aho-Corasick. Two architectural choices, however, contrast with the present POSIX Threads approach: the authors do not explicitly describe how the input stream is partitioned across the four worker threads nor how cross-boundary matches are handled, and their speedup is tightly coupled to platform-specific SIMD intrinsics rather than to portable data-parallel chunking with overlap. This positions FHBM as a complementary, automaton-restructuring strategy whose throughput gains stem from microarchitectural exploitation rather than from explicit input partitioning, providing a useful comparative reference point for the present Pthreads-based design.

### 3.3.3 An Optimized Parallel Failure-less Aho-Corasick Algorithm for DNA Sequence Matching

Thambawita et al. [5] investigate the throughput limitations of the canonical Aho-Corasick (AC) algorithm in bioinformatics workloads, where the dictionary may contain thousands of DNA patterns and the input may reach hundreds of megabytes per scan. The authors observe that, although the Parallel Failure-less Aho-Corasick (PFAC) library of Lin et al. already exposes massive thread-level parallelism by launching one thread per input character, it remains a domain-agnostic implementation whose 256-entry transition table and dispersed memory layout do not exploit two structural properties of DNA matching: the alphabet contains only four symbols ($\{A, T, C, G\}$) and the matching kernel has strong spatial and temporal locality. As a result, the baseline PFAC implementation underuses memory bandwidth and has poor cache behaviour on the GPU memory hierarchy.

To overcome these limitations, the authors propose an application-specific optimization of PFAC on a NVIDIA Fermi GPGPU with three coordinated changes. First, the AC transition table is reduced from a generic $256 \times N$ matrix to a compact $4 \times N$ table aligned with the DNA alphabet, drastically shrinking the working set. Second, the transition table is mapped onto texture memory to exploit its two-dimensional spatial caching, while the input text is staged into shared memory to exploit temporal locality across the threads of each block. Third, the `nextState` and `matchedPatternId` arrays are merged into a single contiguous one-dimensional array so that consecutive accesses benefit from cache-line reuse. The underlying parallel decomposition is preserved from PFAC: every input character is assigned its own thread, which independently traverses the failure-less automaton until a mismatch terminates its scan, and matches are recorded in thread-local result slots that require no inter-thread synchronization during the search phase.

The evaluation is conducted on a NVIDIA Tesla C2075 (Fermi architecture), using five synthetic DNA dictionaries (ranging from $1{,}000$ to $5{,}000$ patterns) and five synthetic DNA input datasets sized between 76 MB and 380.2 MB. The reported metric is wall-clock execution time, measured against the unmodified PFAC library; absolute throughput in MB/s and any sequential CPU baseline are absent. Across all configurations, the optimized implementation consistently outperforms the original PFAC, with the largest datasets exhibiting approximately a $3\times$ speedup over the baseline library. The authors further observe that memory-layout optimizations – in particular the merged one-dimensional array and the use of texture memory – have a substantially larger impact on performance than tuning L1 cache sizes or per-block thread configurations.

This study reinforces two principles that are directly transferable to the present Pthreads-based design even though the target hardware differs: first, that the read-only nature of the AC automaton after construction enables fully lock-free, thread-local matching, irrespective of whether parallelism is realized on a GPU or on a multi-core CPU; second, that the placement of the automaton and the input within the cache hierarchy often matters more than raw thread-count scaling. Two limitations, however, restrict the direct applicability of these findings to the current work. The PFAC decomposition launches one thread per input byte, which is well suited to GPU SIMT execution but mismatched with the much coarser granularity of multi-core CPUs, where the Pthreads model favours chunk-based data partitioning with overlap regions of $\texttt{max\_pattern\_len} - 1$ bytes. Moreover, the alphabet-specific reduction to a $4 \times N$ transition table relies on the small DNA alphabet and cannot be reused for the full 256-symbol byte alphabet of IDS/YARA workloads targeted by the present implementation. The Thambawita et al. study thus serves as a complementary point of comparison, framing PFAC-style fine-grained parallelism as the GPU counterpart of the coarse-grained, chunked Pthreads strategy adopted in this thesis.

### 3.3.4 The Diversification and Enhancement of an IDS Scheme for the Cybersecurity Needs of Modern Supply Chains

Deyannis et al. [20] examine the performance and energy cost of software Aho-Corasick matching when an industrial supply chain node must concurrently execute its primary business workload and inspect real-time network traffic. The authors observe that the canonical Snort IDS, although well-tuned, saturates the CPUs of low-power supply chain assets and therefore introduces a deployment gap: edge nodes either drop packets under load or shift inspection costs upstream, which conflicts with the distributed perimeter defence recommended by recent supply chain cybersecurity frameworks. The bottleneck is clearly in the multi-pattern matching kernel, which Snort implements with Aho-Corasick.

To restore line-rate operation without altering the IDS configuration model, the au-

thors offload the AC engine to the FPGA fabric of a low-cost PYNQ Z1 board (Xilinx Zynq-7000 MPSoC). The fully expanded, failure-free DFA is serialized as a flat two-dimensional integer array indexed by `dfa[state * 256 + char]`, eliminating pointer-chasing and turning every transition into a single arithmetic lookup – a layout structurally identical to the read-only `goto_tbl` adopted by the present Pthreads implementation. The transition table is then fed into a four-stage hardware pipeline implemented in Vivado HLS and clocked at 410 MHz, while the official Snort software stack continues to run on the co-located ARM Cortex-A9 cores; only the AC matching kernel is replaced. Because the design consumes less than 10%

The evaluation employs four publicly available IIoT traffic datasets – the Malware Traffic Analysis blog capture, DARPA-99, the UNSW ToN_IoT dataset and the NCC Group honeypot trace – replayed against the same Snort ruleset, comparing the FPGA-accelerated build against a fully optimized software Snort baseline running on the identical PYNQ board. The reconfigurable design delivers $1.2\times$ to $1.4\times$ acceleration in pure execution time (e.g., $79.4$ s versus $108.9$ s on the DARPA-99 trace) and up to $1.4\times$ higher packet-per-second rates on traces dominated by large packets, while underperforming the software baseline by approximately 10%

In sum, the paper confirms that the matching phase of Aho-Corasick is the dominant cost of signature-based IDS in resource-constrained settings and demonstrates that a hardware-pipelined DFA can recover throughput at a fraction of the energy budget. Three observations transfer directly to the present Pthreads-based design: the use of a fully expanded failure-free DFA stored as a flat $N \times 256$ integer array is equally appropriate for shared-memory CPU traversal, since it preserves $O(1)$ per-character transitions while permitting concurrent read-only access by multiple worker threads; the matching phase rather than the automaton construction dominates the cost, justifying the architectural decision to parallelize only the search loop; and packet-size or input-size sensitivity is a workload effect that any chunk-based parallel implementation should evaluate, since small inputs amortize less of the per-chunk start-up cost. The principal divergence is architectural: while the paper realizes parallelism through hardware pipelining and replicated FPGA IP cores, the present Pthreads design pursues coarse-grained data parallelism through chunked partitioning of the input with overlap regions of `max_pattern_len` $- 1$ bytes, exploiting commodity multi-core CPUs and avoiding the reconfigurable-hardware deployment cost.

### 3.3.5 Fast String Searching on PISA

Jepsen et al. [21] target the bandwidth and per-byte compute bottlenecks that arise in disaggregated data centres, where storage nodes carry weak CPUs and every byte of incoming data must traverse PCI-Express to reach a general-purpose processor. The authors observe that pre-filtering large volumes of stored or in-flight data with multi-

pattern string matchers is essential to keep downstream analytics manageable, but that an x86 host inspecting every byte does not scale to disk-array or line-rate workloads. Aho-Corasick is the standard algorithm for this pre-filter, and the paper poses the question of whether its deterministic finite automaton can be executed entirely within the data plane of a programmable network ASIC, exploiting native pipeline parallelism rather than thread-level parallelism on a CPU.

To answer this question, the authors introduce PPS, a P4-based compiler and runtime that maps an Aho-Corasick-derived DFA to the PISA architecture of the Barefoot Tofino switch. Two transformations are applied to the canonical automaton. First, a $k$-stride DFA is created to process $k$ bytes at a time, so that each pipeline pass consumes several characters at the cost of a multiplicatively larger transition table; this trades on-chip SRAM for higher per-stage throughput. Second, the resulting DFA is optionally partitioned into up to three independent sub-automata that are placed in different pipeline stages and traversed in parallel against the same packet, allowing the aggregate pattern set to exceed the memory of any single stage. To enforce correctness when packets are processed independently, the system mandates that adjacent input chunks overlap by the length of the longest pattern – a constraint conceptually identical to the $\texttt{max\_pattern\_len} - 1$ overlap adopted by the present Pthreads implementation, although applied here at the network MTU granularity. An optional CRC16 hash representation of the transition table is offered as an accuracy-for-memory trade-off; this approximation is incompatible with exact-match requirements and is therefore restricted to use cases tolerant of false positives.

The evaluation contrasts PPS running on a Tofino switch against two CPU baselines – Spark SQL on a four-node dual-socket Intel Xeon E5-2603 cluster and Unix `grep` on a 12 GB RAM-disk log – using Snort community-rule `content` patterns of up to 32 characters and three corpora: the 4.7 GB Podesta e-mail archive, a 212 GB Twitter streaming capture, and the synthetic log file. End-to-end execution times measured at 128 patterns yield a $> 20\times$ speedup over Spark SQL on the e-mail workload and a $6.5\times$ speedup on the tweet workload (5.43 min versus 35.4 min), while throughput at 100 Snort patterns with stride 4 is calculated – not measured – at $3.8$ Tbps for a 64-port Tofino. The authors are explicit that the Tbps figure is a theoretical upper bound derived from the deterministic line rate of the compiled P4 program; CPU baselines are measured end-to-end, and external GPU/FPGA references (e.g., 150 Gbps on GPU, 45 Gbps for DFC on x86) are quoted rather than reproduced.

The paper shows that Aho-Corasick is an algorithm that scales from a single CPU core all the way to multi-Tbps switching silicon, and it makes two design decisions that map directly onto the present Pthreads-based work. The mandatory overlap of *longest-pattern* bytes between adjacent input chunks is the same correctness primitive that the present implementation enforces by reserving $\texttt{max\_pattern\_len} - 1$ warm-up bytes at

every chunk boundary; this independent confirmation is particularly valuable because it derives from an entirely different parallel substrate, showing that the overlap is not just an artifact of thread-level parallelism. Likewise, DFA partitioning provides a precedent for splitting a pattern set across independent execution units, although the present work partitions the *input* rather than the automaton, leaving the goto table intact for shared, read-only access by all Pthreads workers. The principal divergence is the hardware target: PPS realizes parallelism through replicated pipeline stages and TCAM/SRAM primitives in a switch ASIC, whereas the present design pursues coarse-grained data parallelism on commodity multi-core CPUs, accepting lower theoretical throughput in exchange for portability and exact matching.

### 3.3.6 Practical Multi-Pattern Matching Approach for Fast and Scalable Log Abstraction

Tovarňák [22] addresses the throughput limitations of multi-pattern matching in log-abstraction pipelines, where each incoming log line must be classified by matching it against a potentially large set of regex-like patterns. The author observes that the naive strategy of iterating over the entire pattern set per log line incurs $O(|\text{patterns}|)$ cost per input and degrades severely once the dictionary contains more than a few tens of entries, while general-purpose multi-regex libraries such as Google RE2 hit memory-cap limits when the DFA representation exceeds a few hundred patterns. The work is positioned alongside Aho-Corasick as a multi-pattern matcher built on a trie-based automaton, but it explicitly targets the same parallelism model adopted by the present thesis: a single automaton constructed once and shared, read-only, across multiple concurrent workers on a commodity multi-core CPU.

To recover throughput, the author proposes REtrie, a compressed trie that combines literal-character pointers with regex *token* pointers (named wildcards) and supports controlled backtracking only over the token segments, preserving an $O(|\text{input}|)$ matching cost in the typical case. Three design choices map directly onto the present Pthreads design: the trie is constructed sequentially and then declared immutable, so that worker processes need no synchronization during the matching phase; node arrays are compressed with PATRICIA-style path compression and two-level adaptive nesting, lowering the memory footprint of the otherwise sparse 256-entry pointer arrays and improving cache locality; and data-level parallelism is realized by routing disjoint subsets of input log lines to independent Erlang OTP processes, each of which traverses the same shared automaton without any inter-process locking. The system is implemented in Erlang with native-code generation via the HiPE compiler, so the parallelism model is process-based rather than thread-based, but the architectural invariant – read-only shared automaton, thread-local results, no locks on the hot path – is the same one adopted by the Pthreads design of the present thesis.

The evaluation is conducted on a dual-socket Intel Xeon E5-2650 v2 (2.60 GHz) with 64 GB of RAM, scaled from one to eight cores. The pattern sets include a hand-curated set of 22 syslog `auth/authpriv` patterns ($P_{22}$) and large Apache Hadoop sets ($R_x$) with up to 3,500 entries derived from source-code analysis. The runtime workloads include $D_0$, a real-world capture of 100 million syslog lines collected over one month, plus partially synthetic derivatives ($D_x$, $H_x$) of 1.1 million messages each constructed by controlled duplication and shuffling. Each measurement is averaged over ten runs with extreme values discarded. The single-core comparison shows that REtrie's running time remains essentially constant as the pattern count grows from 1 to 22, whereas the naive baseline reaches roughly 12 s on $P_{22}$ versus $\approx 3$ s for REtrie and RE2; for $R_x \geq 2{,}000$ patterns the Erlang REtrie even surpasses C++ RE2, which collides with its default DFA memory limit. The multi-core scaling on $D_0/P_{22}$ exhibits a near-ideal $2\times$ speedup at two cores; at eight cores the 100-million-line workload is processed in $52.543$ s, equivalent to $\approx 1{,}903{,}203$ log lines per second, with the speedup growth flattening past four cores, in line with the ceiling expected from Amdahl's law [11] and from memory bandwidth.

Taken together, the paper provides independent empirical validation of two design choices that underpin the present Pthreads-based implementation of Aho-Corasick. First, the architectural invariant of a read-only shared automaton accessed concurrently by multiple worker processes is shown to deliver near-ideal scaling at low core counts on commodity multi-core hardware, with the diminishing-returns regime located around four to five cores, an observation consistent with the early plateau found in the present experiments. Second, the use of path compression and adaptive node sizing to improve cache locality reinforces the importance of the flat `goto_tbl` layout employed by the present implementation, even though the algorithmic context differs. Two limitations restrict the direct transferability of the results. The REtrie automaton is not an Aho-Corasick DFA: it lacks the failure and dictionary-suffix functions and processes each log line as a single bounded record, so the chunk-boundary overlap problem that motivates the present implementation's `max_pattern_len` $- 1$ warm-up region simply does not arise here. Moreover, Erlang OTP scheduling abstracts away the cache, false-sharing and NUMA effects that the Pthreads design must address explicitly. The Tovarňák study therefore serves as a precedent for the parallel *strategy* (lock-free shared immutable automaton) rather than for its concrete realization, and as a useful throughput reference point on comparable Xeon-class hardware for multi-pattern matching at the line-of-text granularity.

### 3.3.7 Pattern Matching in YARA: Improved Aho-Corasick Algorithm

Regéciová et al. [23] address a particular failure mode of the canonical Aho-Corasick implementation embedded in the YARA rule-matching framework, a representative pro-

duction scanning tool that relies on this algorithm. The authors observe that YARA's preprocessing phase extracts short literal substrings – termed *atoms* – to seed the AC automaton, but the atom-selection heuristic fails to represent the wildcards, character classes and metacharacters that increasingly appear in industrial signatures. As a result, the scanner either selects empty atoms, in which case the matching loop degenerates into a naive byte-by-byte comparison, or it issues a "slowing down" warning that renders the rule unusable on large-scale platforms such as VirusTotal Hunting. The bottleneck is therefore not in raw matching throughput but in the construction-time choice that determines which literal fragments seed the AC automaton.

To overcome this limitation, the authors propose MYARA, a fork of YARA 4.0.2 introducing three coordinated changes to the preprocessing phase while leaving the matching loop intact and sequential. First, atoms are encoded as 256-bit *bitmaps*, where each bit indicates whether the corresponding byte value is admissible at that position; a literal character, the dot metacharacter and a class such as `[a-z]` are therefore expressed in a uniform structure that supports fast subset and inclusion tests through bitwise operators. Second, the AC `goto` function is generalized to accept *sets* of characters per transition rather than single bytes, and the resulting non-determinism is resolved by an explicit determinization step before the failure function is computed. Third, the existing Aho-Corasick Interleaved State Matrix (ACISM) – a flat array of 32-bit integers that encodes both `goto` transitions and failure links for $O(1)$ lookup – is updated to accommodate the denser slot occupancy produced by multi-symbol transitions. Because the resulting automaton remains entirely read-only after construction, the same shared-memory invariant relied upon by the present Pthreads design continues to hold.

The evaluation is conducted on a server equipped with an Intel Xeon E5-2697 v4 at 2.30 GHz running Debian 4.9.168, with binaries built using GCC 8.3.0. Three pattern dictionaries of contrasting composition are used: a hand-crafted set of five YARA rules targeting Bitcoin addresses, e-mail addresses and the INI format; the publicly available ReversingLabs corpus comprising 371 rules with $0$%

On balance, the paper offers two contributions that bear directly on the present Pthreads-based work even though it introduces no parallelism of its own. First, it documents and characterizes the ACISM representation that the present implementation – and any other shared-memory parallel Aho-Corasick variant – can adopt as a strictly read-only substrate, with the additional benefit that bitmap-encoded transitions remain compatible with the flat `goto_tbl[state * 256 + byte]` layout adopted in this thesis. Second, it provides documented single-threaded baselines on a 8.54 GB realistic corpus that serve as external reference points for the present evaluation. Three limitations restrict the direct applicability of the results, however: the optimization is concentrated in the construction phase, whereas the thesis specifically targets the matching phase; the reported gains evaporate on rulesets dominated by literal patterns (the ReversingLabs

case), which closely resemble the literal-dominated Snort and Emerging Threats rule sets exercised in the present evaluation; and the absence of throughput measurements in MB/s precludes a direct quantitative comparison. The Regéciová et al. study therefore complements rather than competes with the present work: it improves the automaton fed to the scanner, while this thesis improves the parallel execution of the scanner against an unchanged automaton.

### 3.3.8 SIMD-Accelerated Regular Expression Matching

Sitaridi, Polychroniou and Ross [7] focus on the sequential bottleneck that arises when a DFA-based pattern matcher is embedded in an in-memory database column-filter operator: the per-character transition $s_{t+1} = \delta(s_t, c_t)$ creates a tight data dependency that defeats straightforward SIMD auto-vectorization, leaving most lanes of a wide vector register idle. The authors explicitly note that Aho-Corasick is equivalent to a minimal DFA once the failure transitions are absorbed into the transition table, so the analysis of vectorized DFA traversal performance transfers directly to the matching phase of any AC engine. Their focus is therefore the same matching-phase throughput targeted by the present thesis, but addressed at a different level of the parallelism hierarchy.

To exploit the unused lanes, the paper introduces a non-lockstep, short-circuit SIMD-DFA traversal algorithm. Rather than advancing all $W$ lanes of the SIMD register in unison one character at a time – which wastes work whenever a string is rejected or accepted early – the algorithm tracks an independent input offset and DFA state per lane, uses SIMD *gather* instructions to fetch the relevant byte for each lane in a single instruction, and replaces any lane that reaches a sink state with a fresh input through selective loads. Three auxiliary techniques sustain the throughput: an eight-byte buffered gather amortizes the cost of non-contiguous loads, a two-way unrolled inner loop hides instruction latency, and a branch-free selective store records the identifiers of accepted strings. Critically for the present discussion, the authors compose this SIMD-level data parallelism with thread-level parallelism by spawning one worker per hardware thread; the DFA transition table is shared, read-only, across all workers, and string offsets are partitioned among them. This is structurally identical to the present Pthreads design, with the difference that the thesis partitions a *single* large input into chunks, whereas the paper partitions a column of many short strings.

The evaluation contrasts two platforms: an Intel Xeon E3-1275v3 with four Haswell cores and 256-bit AVX2, and an Intel Xeon Phi 7120P with 61 cores and 512-bit SIMD. Pattern dictionaries include a 90-state URL-validation regex (23 KB transition table), a 9-state e-mail regex, and synthetic multi-pattern dictionaries built from random English words whose DFA size is swept from $10$ to $10^5$ states to cross the L1, L2 and last-level cache boundaries. Inputs are synthetically generated string columns of varying

length, selectivity and average failure point. Throughput is reported in GB/s of total bytes scanned together with the fraction of peak DRAM bandwidth achieved. The non-lockstep algorithm yields up to $2\times$ over a scalar baseline on the Haswell CPU and up to $5\times$ on the Xeon Phi, with the gap over lockstep reaching $3\times$ for long strings that reject early. Two further observations are particularly relevant. First, throughput scales linearly with hardware-thread count on both platforms, confirming that SIMD-level and thread-level parallelism compose without contention as long as the DFA is shared read-only. Second, performance peaks when the DFA fits in the L1/L2 cache: once the transition table grows past last-level cache, vectorization gains shrink, with throughput reverting to a memory-bandwidth-bound regime.

Ultimately, three findings carry directly into the present Pthreads-based work, with the caveat that the underlying parallelism mechanisms differ. First, the explicit observation that AC is equivalent to a minimal DFA, combined with the demonstrated linear scaling of thread-level parallelism over a shared read-only transition table, independently validates the architectural choice of immutable post-construction `goto_tbl` central to the present design. Second, the empirical demonstration that cache residency dominates throughput once the DFA exceeds last-level cache provides a documented mechanism behind the saturation that the present thesis observes on large rule sets, motivating both the cache-locality concerns of the flat `goto_tbl[state * 256 + byte]` layout and the evaluation of memory-bandwidth contention as the thread count grows. Third, the orthogonality of SIMD-level and thread-level parallelism documented in this paper makes it a natural reference for positioning the Pthreads strategy as a baseline that future work could combine with SIMD intrinsics in the inner loop. Two limitations restrict the transferability of the numerical results: the workload is restricted to fixed-length strings in database columns, in contrast to the variable-length chunked file scans targeted by the present thesis, and the cache-residency assumption holds only for the small URL regex used in most experiments and breaks down for the large multi-pattern dictionaries that motivate AC in security applications.

### 3.3.9 Harry: A Scalable SIMD-based Multi-literal Pattern Matching Engine for Deep Packet Inspection

Xu et al. [24] tackle the per-byte cost of multi-literal pattern matching in software Deep Packet Inspection (DPI), which the authors identify as the dominant bottleneck of production stacks such as Snort, Suricata, ModSecurity and CloudFlare's WAF. The canonical Aho-Corasick (AC) algorithm is acknowledged as the traditional choice but is shown to leave large performance margins on the table compared with the SIMD-accelerated FDR engine of Hyperscan; FDR itself, however, requires three SIMD operations per input character regardless of the vector width, attains only $50\%$

Three coordinated techniques constitute the Harry engine. A column-vector-based

shift-or matching algorithm replaces FDR's row-vector decomposition: instead of loading one row per input character, Harry loads one column vector per literal-position slot, so that the number of SIMD operations becomes proportional to the literal length $\ell$ (fixed at eight in the implementation) rather than to the input length, yielding only $0.41$–$0.71$ operations per input character on AVX-512 and raising vector utilization to $87.5\%$

Harry is evaluated on an Intel Xeon Gold 6348 (2.60 GHz, two sockets of 28 cores each, AVX-512) running Ubuntu 20.04, with GCC 9.4.0 used without optimization flags. Literal rules are extracted from two production rule sets – OWASP ModSecurity Core Rule Set v3.3.2 and Snort Emerging Threats v2.9.0 – and partitioned into nine subsets spanning $10$ to $3{,}000$ rules. Three input traces are used: $121$ MB of real IXIA HTTP packets, $4$ MB of Alexa non-HTTP packets and $763$ KB of synthetic random bytes; each experiment is averaged over $1{,}000$ runs. The reported throughput ranges from $30$ to $70$ Gbit/s and represents a $26\times$–$45\times$ speed-up over canonical Aho-Corasick on real HTTP and non-HTTP traffic, peaking at $52\times$ on the random byte stream, and a $1.06\times$–$2.09\times$ improvement over FDR. The heuristic encoding selector picks the better variant in every evaluated configuration, and Harry maintains its advantage across the full $10$–$3{,}000$ rule range, in contrast to the prior Teddy engine, which degrades sharply beyond roughly sixty rules.

Overall, the paper has two effects on the related-work positioning of the present thesis. First, it concretely quantifies the ceiling that single-threaded SIMD literal matching can reach on commodity AVX-512 hardware – $30$–$70$ Gbit/s at $0.41$ operations per character with $87.5\%$

### 3.3.10 Characterizing Realistic Signature-based Intrusion Detection Benchmarks

Aldwairi, Alshboul and Seyam [6] address a methodological gap that affects every empirical study of pattern matching for signature-based Intrusion Detection Systems (NIDS), including the present thesis: there is no agreed standard for the datasets, the metrics or the simulation environment under which Aho-Corasick and competing algorithms are evaluated, and the resulting heterogeneity makes cross-paper comparisons unreliable. The authors review the most cited public datasets – DARPA, KDD Cup 99, NSL-KDD, Kyoto 2006+ and CAIDA – and document that all of them were designed for anomaly-based detection and therefore lack representative attack signatures; privately collected traces, conversely, cannot be redistributed for legal reasons. The paper therefore positions itself not as a parallelization contribution but as the construction of a reproducible NIDS benchmark stack that subsequent works – this thesis included – can adopt to ground throughput and speed-up claims in realistic conditions.

The contribution is delivered as a five-part artifact. First, a GUI-based parser ingests heterogeneous IDS rule formats and extracts the `content` and `uricontent` keywords

of 120 Snort rule files, yielding hexadecimal-encoded signature dictionaries of varying cardinality (from 300 to over 2,400 signatures, suitable for scalability sweeps). Second, eight representative traces are curated from the 78 DEFCON17 Capture-the-Flag captures: four “good” traces (audio-video streaming, downloads, Hotmail, live chat) representing baseline traffic, two “ugly” traces (51 and 58) where 45%

The hardware platform is not specified; the engine is implemented in C# and the authors flag that the runtime’s garbage collector limits the precision of memory measurements. The reported numbers are nevertheless useful as a sequential baseline. Aho-Corasick is the slowest of the three viable algorithms on the large “audio-video” trace, taking $2{,}374{,}992$ µs against Wu-Manber’s $619{,}469$ µs, and consumes the largest memory footprint, peaking at $6{,}840{,}694$ bytes versus $2{,}694{,}016$ bytes for KMP. The runtime-versus-signature-count sweep on the same trace shows that AC scales non-linearly: the time grows from $168{,}079$ µs at $300$ signatures to $2{,}374{,}992$ µs at the full dictionary, an effect the authors describe as “almost exponential”. The memory-versus-packet-count sweep on the “ugly” trace 58 reports a similar growth – from $29{,}402$ bytes at $25{,}000$ packets to $3{,}871{,}408$ bytes for the full trace – although this expansion most likely reflects accumulated match-record storage and C# allocation behavior rather than an intrinsic growth of the AC automaton, whose DFA trie is fixed by the signature set.

In sum, the paper provides the present thesis with two contributions that no other reviewed work delivers in combination. First, the DEFCON17 trace selection and the Snort-derived signature dictionaries are documented, redistributable benchmark inputs that can be replayed against the present Pthreads implementation to expose the multi-core speed-up under the same workload conditions used elsewhere in the literature. Second, the per-trace sequential AC runtimes – in the order of tens of milliseconds for the smaller traces and a few seconds for the largest – furnish numerical reference points against which the present multi-thread throughput can be situated. Three limitations restrict the direct use of the published numbers. The hardware on which they were measured is not specified, so absolute runtimes cannot be reproduced. The implementation language (C#) and its garbage collector make memory figures imprecise. And the paper does not report throughput in MB/s or Gbps, which are the metrics the present thesis adopts; conversion from microseconds plus payload size in bytes is therefore required when quoting the Aldwairi figures as competitors. The paper should be cited for benchmark methodology and realistic-workload selection rather than as a parallel-AC technique, and it complements the multi-core, multi-threaded contributions reviewed above by anchoring the evaluation in the canonical NIDS context that motivates Aho-Corasick parallelization in the first place.

## 3.4 SYNTHESIS AND IDENTIFIED GAP

The systematic review answered both research questions by identifying the predominant parallelization strategies for Aho–Corasick on multi-core CPUs and the optimization techniques most frequently employed in practical applications.

- **Parallelization strategies:** read together, the ten primary studies cover a broad execution space. Some approaches target non-CPU substrates: the GPU implementation of Thambawita et al. [5], the FPGA fabric used by Deyannis et al. [20], and the programmable switch ASIC design of Jepsen et al. [21]. Others remain on general-purpose processors but adopt coordination models other than native shared-memory threading, such as MPI in Qu et al. [19] or Erlang OTP processes over a non-Aho–Corasick automaton in Tovarňák [22]. These studies confirm that Aho–Corasick matching can be parallelized, but they do not directly characterize a portable POSIX Threads implementation over the unmodified automaton.

- **Memory and automaton optimizations:** a second group of contributions attacks the memory cost of the automaton itself. Lee and Yang [2] split the automaton into head and body regions to fit the hot portion into cache, while Sitaridi et al. [7] and Xu et al. [24] use SIMD-oriented literal matching to reduce the cost of state transitions. Regéciová et al. [23] focus on parallel construction while leaving matching sequential. Across these papers, the recurring observation is that throughput saturates once the transition table escapes the last-level cache; however, none isolates and characterizes that memory-bound regime as the primary object of study.

- **Applications and benchmarking:** the application studies anchor the problem in realistic security and inspection workloads. Deyannis et al. [20] and Jepsen et al. [21] motivate line-rate intrusion detection on constrained or specialized platforms, while Regéciová et al. [23] connects the automaton to YARA-style malware scanning. Aldwairi et al. [6] contributes benchmark methodology and redistributable traces rather than a parallel technique. These application papers justify the use of realistic dictionaries and corpora, but they do not close the shared-memory CPU gap identified above.

The gap addressed by the present work is therefore the following: a portable, POSIX Threads parallelization of the *unmodified* Aho–Corasick automaton on commodity shared-memory multi-core CPUs, based on input-text partitioning with a `max_pattern_len` $-$ 1 overlap and a strictly read-only transition table, evaluated specifically in the regime where the automaton greatly exceeds the last-level cache and with exact-match equivalence to the sequential baseline. No reviewed study

delivers this combination. Based on these findings, the next chapter presents the design and implementation of the proposed shared-memory parallel version of the Aho–Corasick algorithm.


4

# 4 PROPOSED SOLUTION

This chapter presents the proposed solution: a modular parallel framework for the Aho-Corasick multi-pattern matching algorithm, targeting shared-memory multi-core CPUs. The design is guided by a single structural decision from which every other property follows. The design choices presented in this chapter are directly motivated by the findings of the systematic literature review discussed in Chapter 3: prior work shows that input partitioning with boundary handling is the natural axis for matching-phase parallelism [19], while the memory traffic of large AC-DFA tables motivates both a read-only flattened automaton and explicit cache-aware experiments [2], [7]. The default execution is split into separate phases: the master thread builds the automaton, after which its state-transition tables are declared strictly read-only. An optional construction variant parallelizes the failure-link traversal, but in every case the completed automaton is immutable before scanning begins. Because the automaton is never mutated during scanning, multiple worker threads can traverse it concurrently without any mutual-exclusion primitive on the hot path. This lock-free property is what allows the parallel search to approach the limits imposed by the hardware rather than by software contention.

The framework is organized so that distinct optimization ideas can be plugged in and measured independently. Two complementary forms of scan parallelism are considered: partitioning the *input text* among threads, and partitioning the *pattern dictionary*. In addition, an algorithmic optimization of the match-emission step is proposed that benefits both the sequential and the parallel cases, and a build-phase optimization is evaluated separately. The remainder of this chapter describes each component. The overall phased execution flow of the proposed parallel framework is illustrated in Figure 5.

## 4.1 DATA PARTITIONING AND BOUNDARY OVERLAP

To exploit thread-level parallelism on a single large input, the framework partitions the text buffer into contiguous segments, one per worker thread. Each worker traverses the shared automaton over its own segment and records matches in a private list, so that no synchronization is required while scanning.

However, a naive partitioning of the byte stream would miss any pattern that crosses

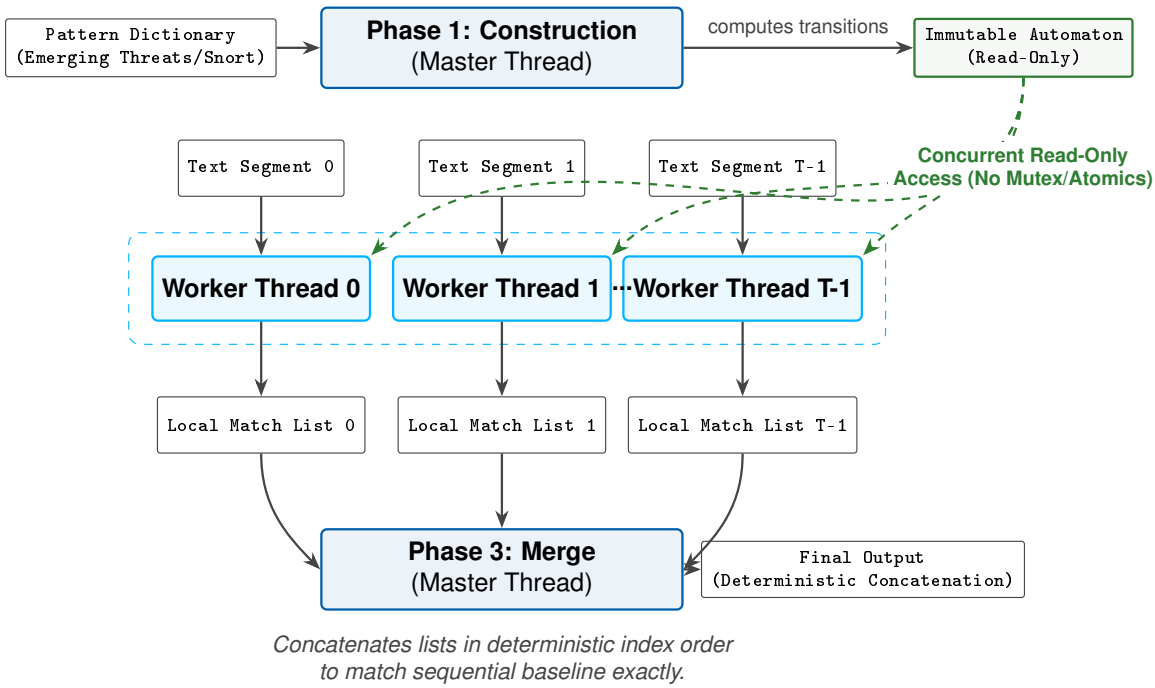



Figure 5: Parallel execution pipeline of the proposed framework, divided into three phases: sequential construction of the read-only automaton (Phase 1), concurrent parallel search across text segments (Phase 2), and sequential deterministic merge of match lists (Phase 3).

the boundary between two adjacent segments, because neither worker would observe the full pattern. To prevent these boundary false negatives, the design uses a controlled overlap. Let $L_{\text{max}}$ be the length of the longest pattern in the dictionary. A worker responsible for the segment spanning offsets $[S, E)$ does not begin scanning at $S$ but at $S - L_{\text{max}} + 1$, except for the first worker, which starts at offset $0$. This warm-up margin guarantees that, by the time the worker reaches the start of its owned region at $S$, the automaton is in the same state it would have reached in a single sequential pass from offset $0$, so every boundary-crossing pattern is observed by exactly one worker. Any match whose end position falls before $S$ is suppressed, because it is owned and reported by the previous worker. Conversely, assigning each match to the segment containing its end position keeps ownership disjoint. This preserves an exact one-to-one correspondence with the sequential baseline and avoids duplicate detections. An overlap of $L_{\text{max}} - 1$ is the minimum that achieves this property; any smaller value loses matches, and any larger value only adds redundant work. This boundary-crossing detection and overlap logic is illustrated in Figure 6.

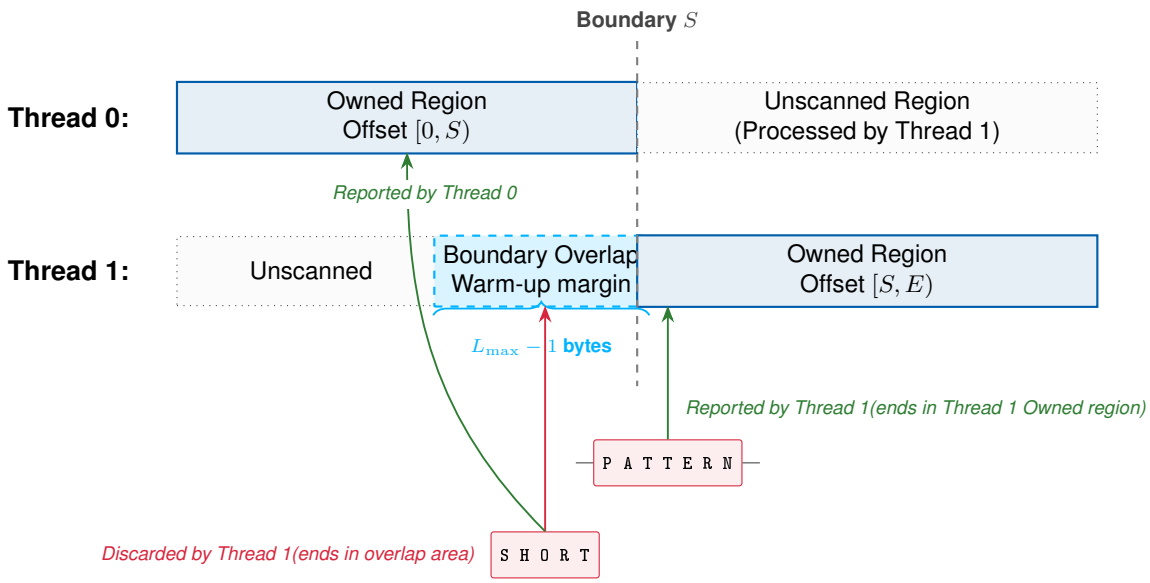



Figure 6: Data partitioning and boundary overlap scanning layout, showing matching ownership and warm-up margins across Thread 0 and Thread 1.

## 4.2 LOAD BALANCING ON HETEROGENEOUS CORES

Uniform segments are optimal only when all cores are equally fast and the work is evenly distributed along the text. On a hybrid processor, where Performance and Efficiency cores run at different clock frequencies, equal-sized segments cause the threads on slower cores to finish late, leaving the faster threads idle at the final synchroniza-

tion barrier. The proposed framework therefore includes a topology-aware partitioning strategy that detects the heterogeneous layout and scales each segment in proportion to the relative speed of the core processing it. The goal is to equalize the finishing times of all workers, reducing the idle time wasted at the barrier.

This proportional partitioning is computed once during initialization. The implementation reads the online CPU topology from the operating system, including the sibling list of each logical CPU and the maximum frequency reported by the per-CPU `cpufreq` metadata. Workers are assigned first to physical-core leaders ordered by descending maximum frequency and then to simultaneous multithreading (SMT) siblings; each worker is pinned to its assigned CPU with `pthread_setaffinity_np`. If worker $i$ is assigned to a CPU with maximum frequency $f_i$, its owned text segment is sized as $|s_i| = N \cdot f_i / \sum_j f_j$, rounded to cache-line boundaries, while the same $L_{\max} - 1$ overlap described above is preserved at segment borders. On a homogeneous processor all $f_i$ values are equal, so the scheme degenerates to ordinary static chunking with explicit affinity. If the required operating-system metadata are unavailable, the implementation falls back to equal chunks and identity affinity, preserving correctness without relying on topology information. In this thesis the policy is evaluated only as a heterogeneous-core diagnostic; it is not the source of the canonical performance figures, which come from the homogeneous workstation.

As an alternative that requires no knowledge of the topology, the framework also supports dynamic dispatch, in which the text is divided into many small tasks that idle workers claim through a single atomic counter, so that faster cores naturally process more tasks. Because idle workers simply claim the next available task, the same mechanism also compensates workers slowed by the content itself — for example, by a match-dense region of the text — and not only workers placed on slower cores; the two causes of imbalance are evaluated separately in the results chapter. Figure 7 contrasts the three partitioning policies and the idle time each leaves at the barrier.

## 4.3 FLATTENING THE MATCH-EMISSION PATH

In the canonical Aho-Corasick algorithm, each accepting state references the patterns that end there through linked structures, and reporting all matches at a given state requires walking these pointer chains. With a large dictionary, these pointers are scattered across an automaton much larger than the cache. As a result, each emission causes cache misses and serialized pointer dereferences that compete with the hot state-transition traffic.

To remove this cost, the design proposes a flattened output layout. During construction, the patterns associated with every state are collected into a single contiguous array, and each state stores only a starting offset and a count into that array. At scan time, emitting the matches of a state becomes a sequential read of a short, contiguous

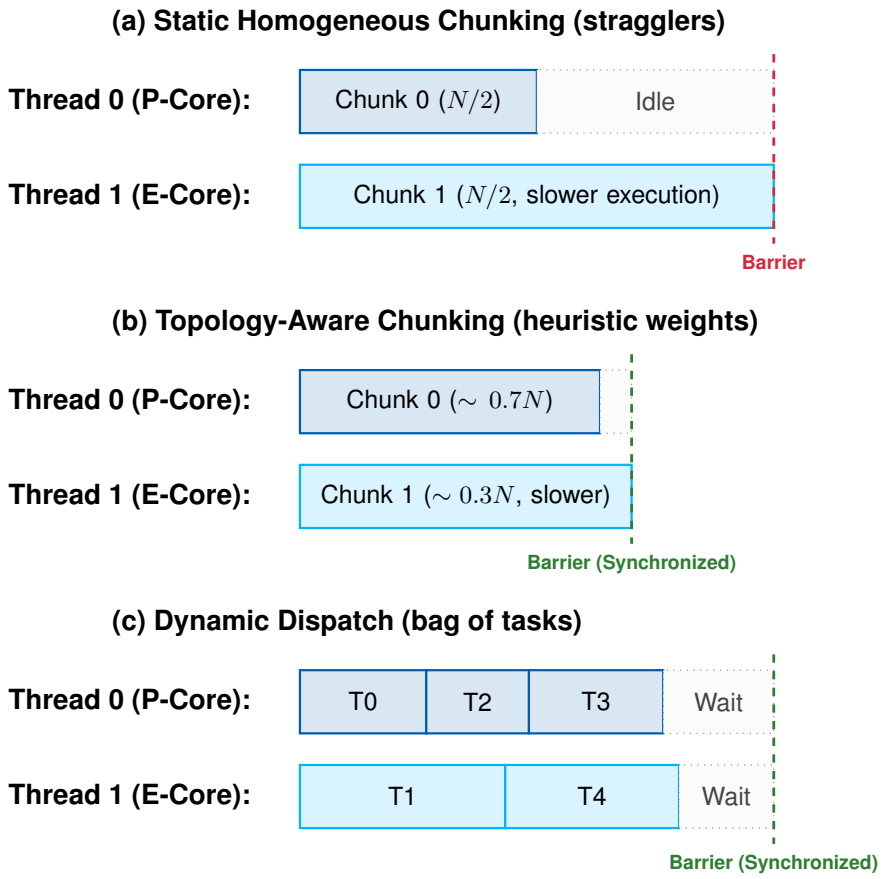



Figure 7: Partitioning policies on heterogeneous cores. Bar length represents elapsed execution time, while labels indicate assigned text size or task identity: (a) static homogeneous chunking leaves the faster Performance core idle at the barrier (stragglers); (b) topology-aware, frequency-weighted chunking sizes each segment to its core's speed; (c) dynamic dispatch hands out small tasks on demand. Both (b) and (c) equalize finishing times.

range. The hardware prefetcher handles this efficiently, removing pointer dereferences from the inner loop. This optimization is purely a memory-layout change: it preserves the exact set of reported matches and is inherited by every scanning variant, whether sequential or parallel. Figure 8 contrasts this contiguous layout with the canonical linked list structure.

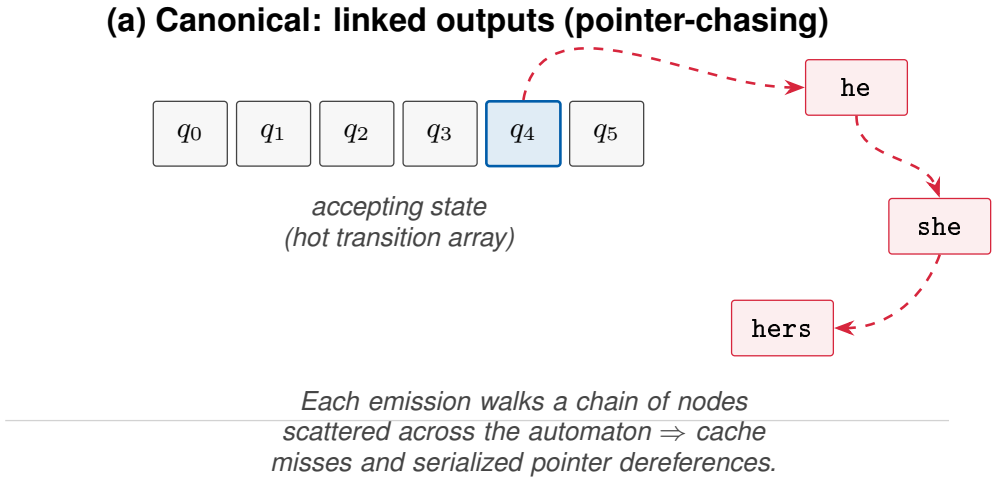



Figure 8: Comparison of match-emission layouts: (a) canonical linked outputs using pointer-chasing across scattered memory nodes, and (b) proposed flattened output layout storing contiguous pattern indices.

The scan-phase strategies above assume that the automaton has already been constructed. The framework also includes a build-phase variant that parallelizes the breadth-first traversal used to compute failure links. Trie insertion remains sequential, because each pattern may create or reuse shared prefix nodes. After the trie exists, however, the failure link of a state at depth $d$ depends only on links from shallower depths. All states at the same breadth-first level can therefore be processed concur-

rently, followed by a barrier before the next level begins. The barrier preserves the dependency order of the sequential construction while exposing the independent work inside each level. Figure 9 illustrates this level-synchronous structure.

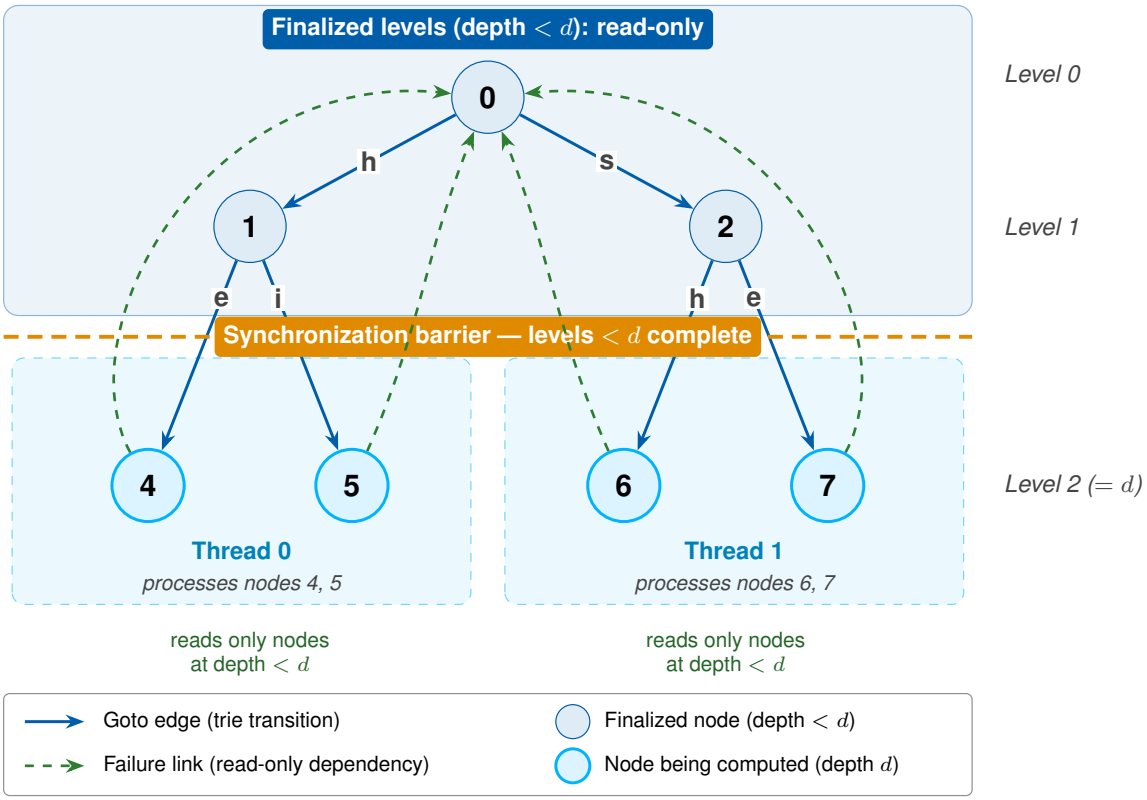
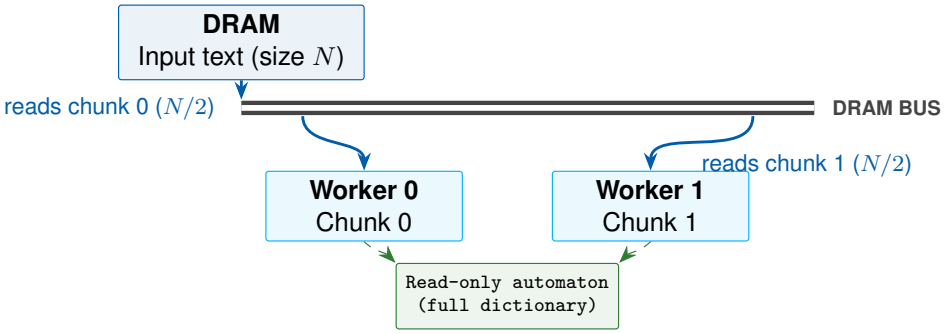



Figure 9: Level-synchronous parallel construction of the failure function. Threads compute the failure links of all states at one breadth-first level in parallel, reading only already-finalized links from shallower levels (green), and meet at a barrier before advancing to the next level.

## 4.4 PARTITIONING THE DICTIONARY

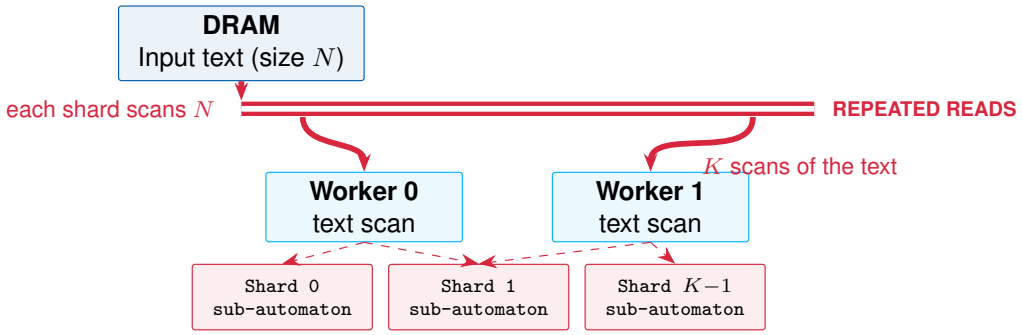
All the strategies above partition the text while keeping the dictionary intact. A complementary approach, evaluated for contrast, partitions the dictionary instead. In this approach, the pattern set is divided into $K$ disjoint groups by prefix bucket, and an independent sub-automaton is built for each group. Each shard therefore owns a disjoint set of global pattern identifiers, so the local match lists can be concatenated after the workers finish; no duplicate-removal step is needed in the merge. This partition is not free, however. The implementation pays the build and memory cost of $K$ sub-automata, including duplicated roots and shared-prefix structure that a monolithic automaton would store only once.

**(a) Text chunking: one shared automaton**




**Total text traffic:** $1 \times N$
Each byte is read once from main memory.
Parallelism comes from dividing the text.

**(b) Dictionary sharding: repeated text scans**




**Total text traffic:** $K \times N$
Shards have disjoint patterns and smaller automata.
The cost is repeated input reads plus duplicated build state.

Figure 10: Text-reading traffic in the two partitioning designs: (a) pure text chunking reads each input byte once while workers share the full automaton; (b) dictionary sharding builds $K$ independent sub-automata and scans the input once per shard, reducing each automaton footprint while multiplying text traffic.

The larger structural cost appears during scanning: each shard must scan the entire input text. Standalone dictionary sharding has no boundary overlap because every worker sees every byte, but it multiplies text traffic by $K$. The motivation is locality, not additional text parallelism: a smaller sub-automaton may fit better in cache and reduce transition-table pressure, but this benefit must compensate for the repeated reads of the same input. The framework also supports a two-dimensional composition with text chunking, in which each pair $(k, n)$ scans shard $k$ over text chunk $n$ with shard-local overlap. Even there, every byte is processed once per shard. Pure text chunking therefore reads each byte once, whereas sharding with chunking multiplies text traffic by the number of shards while reducing the automaton footprint seen by each shard. Whether the locality gained by smaller sub-automata compensates for this repeated input traffic is an empirical question, answered in the results chapter. Figure 10 summarizes this cost trade-off.

# 5

# 5 METHODOLOGY

This chapter describes the experimental methodology used to evaluate the parallel Aho–Corasick scanning strategies introduced in the previous chapter. The evaluation targets three properties of each strategy — search performance, scalability with the number of threads, and correctness with respect to the sequential baseline — under a shared-memory multi-core environment and over pattern dictionaries and text corpora representative of signature-based intrusion detection. Because the workloads of interest produce automata whose footprint far exceeds the last-level cache, the reported figures are expressed as relative quantities (speedup, throughput, and parallel efficiency) rather than as absolute production capacity, so that the conclusions concern scaling behaviour instead of the peak rating of a single machine.

The chapter first establishes the experimental environment and software stack; then characterizes the pattern dictionaries and text corpora and the role each one plays; states the three-phase execution model and the invariants that make the parallelization sound; enumerates the evaluated strategies and the axes along which they compose; separates the performance metrics from the supporting diagnostic instrumentation; defines the per-phase measurement protocol; describes how correctness was established before any performance was reported; and closes with the experimental design, stating each hypothesis, the experiment that tests it, and the section of the next chapter that reports the outcome.

## 5.1 EXPERIMENTAL ENVIRONMENT

The measurement platform, and the source of every headline number, table, and figure in this work, is an AMD Ryzen 9 9950X processor. It is a homogeneous design of the Zen 5 generation: sixteen identical cores, each with two-way simultaneous multithreading (SMT), exposes thirty-two logical hardware threads. Every core reaches the same maximum clock, so the thread pool carries no performance/efficiency asymmetry. Because the cores are uniform, a uniform text partition is naturally load-balanced, and the frequency-weighted, topology-aware weighting introduced in the previous chapter has little to correct on this machine — a point revisited in the load-imbalance comparison against a heterogeneous processor later in this work. The memory hierarchy comprises a private $48$ KiB L1 data cache and $1$ MiB of L2 per core, and a non-unified

64 MiB L3 organized as two 32 MiB slices, one per core complex (CCD); an automaton larger than a single 32 MiB slice therefore cannot remain resident in the L3 of one complex. Main memory is 123 GiB of DDR5, and the aggregate memory bandwidth is the shared resource for which the parallel scans ultimately contend.

The software stack is a purpose-built laboratory written in C11, compiled with GCC 13.3.0 using `-O3 -march=native` and an aggressive warning profile, and linked against the POSIX Threads library [25]. Experiments ran under Linux (kernel 6.17.0, Ubuntu 24.04). To minimize scheduling and frequency-scaling artifacts, the CPU frequency governor was set to `performance`, holding the cores at their sustained all-core clock throughout the sweep; the topology-aware searchers (`pthread_chunked_v3` and `pthread_chunked_v3_flat`) additionally pin each worker thread to a dedicated logical CPU via `pthread_setaffinity_np`, while the remaining searchers rely on OS scheduling without explicit binding. The build was additionally validated under sanitizer instrumentation, namely the Address and Undefined Behavior sanitizers and the Thread Sanitizer, as discussed in Section 5.7.

Internally, the laboratory is organized as a single binary in which each scanning strategy is a self-contained module, registered under a unique name at program start-up and selected at run time. Every strategy therefore executes inside the same program: it receives the same automaton, produced by the same construction code, scans the same memory-mapped corpus buffer, and is timed by the same measurement harness, so that two runs differ only in the strategy under test and in the experimental parameters being varied, such as the thread count. This shared execution path is what makes the comparisons between strategies fair, and its structured per-run output is the source of the database described under metrics and instrumentation below.

Two separate diagnostics appear at the end of the results chapter (Sections 6.9 and 6.10), solely to isolate two load-imbalance effects that the canonical setting cannot exhibit: the imbalance created by heterogeneous cores and the imbalance created by input content whose matches concentrate in one region. Both run on a secondary processor that is not a measurement platform for any of the headline figures reported here.

## 5.2 PATTERN DICTIONARIES AND TEXT CORPORA

The workloads were chosen to sweep the automaton footprint across the cache hierarchy while keeping the input text realistic for the intrusion-detection domain. Table 2 summarizes the pattern dictionaries, which were derived from publicly available Snort and Emerging Threats rule sets, and Table 3 summarizes the text corpora.

The two smaller dictionaries (Snort-100 and Snort-1k) serve as *contrast regimes* that bracket the point at which the automaton stops fitting comfortably in cache; the full Snort dictionary and the Emerging Threats subset are the actual targets of the study,

Table 2: Pattern dictionaries, the resulting automaton footprint, and their experimental role.

| Dictionary | Patterns | States | Footprint | Experimental role |
|---|---|---|---|---|
| Snort-100 | 100 | 1,939 | 1.93 MiB | Smallest automaton; lower-bound contrast regime. |
| Snort-1k | $\sim$1,000 | 11,974 | 11.92 MiB | Intermediate footprint; brackets the throughput cross-over. |
| Snort (full) | 4,188 | 55,479 | 55.23 MiB | Primary target dictionary (moderate footprint). |
| Emerging Threats | 44,678 | 508,896 | 506.78 MiB | Largest footprint; most aggressive memory stressor. |

Derived from the Snort and Emerging Threats rule sets.

Table 3: Text corpora used as scanning input.

| Corpus | Size | Characterization and role |
|---|---|---|
| SimpleWiki | $\sim$1.19 GiB | Natural-language encyclopedic text with low match density; used as the contrast corpus in the cross-corpus experiment, which isolates the effect of the text content. |
| Enron | $\sim$1.32 GiB | Real e-mail messages with medium match density; the canonical benchmark corpus. |
| Enron ($8\times$) | $\sim$10.59 GiB | The Enron corpus replicated eightfold; amortizes the build phase and reduces scheduler noise in the scaling curves. |
| Skew pair (uniform / clustered) | $2 \times 4.00$ GiB | Synthetic control pair for the content-skew diagnostic: identical bytes and identical total matches, differing only in whether the match-dense blocks are spread evenly or concentrated in the first quarter of the file. |

The Enron corpus derives from the public Enron Email Dataset; the skew pair is generated from it as described below.

the latter being the most aggressive stressor, with an automaton far larger than the last-level cache of any platform used here. The footprints in Table 2 are reported as absolute, platform-independent sizes; how each footprint relates to a particular cache level depends on the machine and is therefore interpreted in the next chapter against the cache hierarchy of the measurement platform — the $64$ MiB shared L3, split into two $32$ MiB per-complex slices.

The skew pair deserves a note on construction, because the validity of the content-skew diagnostic rests on a strict invariant. Both corpora are assembled from the same two $8$ MiB building blocks: a *cold* block taken directly from the Enron corpus, and a *hot* variant of that block in which fragments of randomly chosen Snort patterns — between $80\%$ length — are overwritten at random positions until the block reaches a target match density. Each corpus concatenates $512$ blocks, of which $128$ are hot: the uniform variant interleaves one hot block into every four, while the clustered variant places all hot blocks contiguously in the first quarter of the file. The two files therefore contain the same blocks and differ only in block order; the generator verifies that their byte counts and total match counts are identical, so any throughput difference between them is attributable to the spatial placement of the match-dense work — precisely the variable the diagnostic manipulates.

## 5.3 EXECUTION MODEL AND INVARIANTS

Every execution of the laboratory follows three strictly separated phases; this separation is the foundation of the lock-free design.

1. **Construction.** The patterns are inserted into a trie, and a breadth-first (BFS) traversal computes the goto, failure, and dictionary-suffix links, together with the precomputed output structures. The resulting automaton is then declared immutable.

2. **Search.** Each worker thread reads the shared, read-only automaton and its assigned portion of the input text, emitting matches into a private, thread-local list. No mutex or atomic operation is used on the hot path, because the automaton is never modified during this phase.

3. **Merge.** After all workers join, the master thread concatenates the thread-local lists in a deterministic order. This phase is always sequential and accounts for a negligible fraction of the total time, so it is not a target for optimization.

This study respects a set of non-negotiable invariants that make the parallelization sound. The automaton is strictly read-only during the search; the hot loop uses neither locks nor atomics; matches remain thread-local until the join; and the partitioning

preserves match ownership without duplication. For any text-level partitioning, the minimum overlap between adjacent segments is $L_{\max} - 1$ bytes, where $L_{\max}$ is the length of the longest pattern — as established in the previous chapter, the smallest margin that guarantees that a boundary-straddling pattern is detected by exactly one worker. Match ownership is made disjoint by assigning each boundary-crossing match to a single segment according to its end position.

## 5.4 EVALUATED STRATEGIES

The strategies are organized as a set of orthogonal design axes so that each performance effect can be attributed to a single cause. Every variant is a combination of one choice per axis, all selectable at run time within the same binary and all sharing the same automaton. The axes, and which choices are mutually exclusive versus composable, are summarized in Table 4.

Table 4: Evaluated strategies as orthogonal design axes; the work-distribution options are alternatives, while the emission and construction axes compose with any of them.

| **Axis** | **Options (one per run; mutually exclusive within the axis)** |
|---|---|
| Work distribution | Sequential baseline; static homogeneous chunking; topology-aware weighted chunking; dynamic chunk dispatch (bag of tasks); dictionary sharding by pattern prefix; two-dimensional sharding $\times$ chunking. |
| Emission layout | Chained (pointer-chasing) *or* flattened contiguous output table; composes with any work-distribution choice. |
| Construction | Sequential build *or* level-synchronous parallel build (BFS); composes with any work-distribution choice. |

Two of the three axes compose freely. The emission layout (chained pointer-chasing versus a flattened, contiguous output table) and the construction mode (sequential versus level-synchronous parallel build) can be paired with any work-distribution choice and with the sequential baseline, because the parallel build produces an automaton bit-for-bit identical to the sequential one and the emission layout does not change which matches are produced. The work-distribution axis, in contrast, is mutually exclusive: a run partitions the work in exactly one way. The named searchers in the laboratory are therefore compositions of these axes — for example, static chunking with the flat layout, or topology-aware chunking with the flat layout — which is precisely what allows one axis to be varied while the others are held fixed.

The work-distribution options differ in how they assign work to threads. Static homogeneous chunking is the control case: it divides the text into equal segments, one per worker, and adds only the boundary overlap required for correctness. This policy is appropriate when the processor is homogeneous, as in the canonical Ryzen 9 9950X workstation used for the headline measurements.

Topology-aware chunking and dynamic dispatch are two *distinct* responses to the imbalance that equal segments can create on processors whose cores are not equally fast. Topology-aware chunking is a static policy: during initialization it reads the operating system's per-core frequency metadata, assigns each worker to a logical CPU, and sizes the worker's segment in proportion to that CPU's maximum frequency. Faster cores therefore receive larger segments before the scan begins. On a homogeneous processor these frequencies are equal, so the policy collapses to static chunking with explicit thread affinity.

Dynamic dispatch does not use a hardware model. Instead, it splits the text into many small, uniform tasks and lets idle workers pull tasks from a shared atomic counter at run time. Faster cores naturally consume more tasks, so imbalance is corrected reactively rather than by a precomputed segment size. The later heterogeneous-core comparison uses the Intel Core i5-1235U only as a diagnostic P/E case for these policies; it is not part of the canonical measurement platform and supplies none of the headline performance numbers.

The dictionary-parallelism options partition the pattern set instead of the text: prefix sharding builds $K$ independent sub-automata, and the two-dimensional composition combines sharding with text chunking to test whether the two kinds of parallelism add. An exploratory software-prefetch variant of the chunked searcher also exists in the laboratory as a microarchitectural probe; it introduces no new axis and is excluded from the controlled grid and from the reported results.

## 5.5 METRICS AND INSTRUMENTATION

The performance metrics follow the standard definitions introduced in the theoretical framework. Search throughput is reported in MB/s (and, where useful, in Gbps); speedup is the ratio of sequential to parallel search time, as in Eq. 1; and parallel efficiency is the speedup normalized by the thread count, as in Eq. 4. Search time is the quantity these are derived from, and it is always measured separately from construction time, so that the cost of rebuilding the automaton — relevant to rule reloading — is never conflated with scanning throughput.

A second group of quantities does not measure performance but supports its interpretation, and is reported as diagnostic instrumentation rather than as a result. Per-thread timings and the coefficient of variation across repetitions measure load balance and stability, respectively: the per-thread spread exposes synchronization waste at

the final barrier, while the coefficient of variation quantifies how noisy a measurement is rather than how fast it is. Match counts are recorded for every run as a correctness check, not a performance figure (see Section 5.7). Construction (build) time is recorded for the parallel-build experiment. Finally, the laboratory writes every run, together with a snapshot of its environment, to a structured log that is ingested into an SQLite database, which is the substrate for all aggregation and cross-checking; the automaton footprint in bytes is retained there too — not as a headline figure, but as the causal variable used to explain throughput across regimes.

## 5.6 MEASUREMENT PROTOCOL

Each reported data point follows the per-phase repetition protocol summarized in Table 5. Every phase begins with a number of *warm-up* iterations whose times are discarded: the warm-up pays the one-time costs of page faults and cache population so that the subsequent timed iterations measure steady-state behaviour. For each timed series both the mean and the best (minimum) time are kept; the mean is used for the reported throughput, and the dispersion between iterations is retained as the stability indicator described in the previous section.

Table 5: Per-phase repetition protocol of the formal sweep, verified against `sweep.db`.

| Phase | Description | Warm-ups | Timed iters |
|---|---|---|---|
| A | Speedup curves | 2 | 5 |
| B | Footprint sweep | 2 | 5 |
| C | Cross-corpus | 2 | 5 |
| D | Per-thread diagnostic | 1 | 3 |
| E | Parallel construction | 0 | 1 |
| G | Task-granularity sweep | 2 | 5 |

Verified against `sweep.db`.

- **Phase A — Speedup curves.** This phase measures the headline scaling result. It sweeps thread counts $T \in \{1, 2, 4, 6, \ldots, 32\}$ over the canonical Enron corpus and $T \in \{1, 4, 8, 16, 32\}$ over the eight-fold replica, up to the thirty-two hardware threads of the platform, running every parallel searcher against the two single-thread baselines, with two warm-ups and five timed iterations per point.

- **Phase B — Footprint sweep.** Holding the corpus fixed at Enron, this phase varies the dictionary across the four footprints of Table 2 to locate the throughput cross-over, again with two warm-ups and five timed iterations.

- **Phase C — Cross-corpus comparison.** Holding the dictionary fixed at the full Snort set, this phase compares the natural-language SimpleWiki corpus against

Enron, with two warm-ups and five timed iterations, so that any throughput difference is attributable to the corpus rather than to the automaton.

- **Phase D — Per-thread diagnostic.** At the full thread count, this phase records per-thread timings for each parallel searcher — one warm-up and three timed iterations — to quantify barrier idle time and load balance.
- **Phase E — Parallel construction.** This phase compares the sequential build against the level-synchronous parallel build across the four dictionaries; since only the build time is of interest, each build is timed once with no warm-up.
- **Phase G — Task granularity.** Holding the searcher, dictionary, and corpus fixed at the winning configuration (`pthread_dynamic_flat`, full Snort, Enron) at the full thread count, this phase sweeps the number of text tokens per dispatched task, with two warm-ups and five timed iterations, to measure how the atomic-counter contention of the bag-of-tasks scheme trades against load balance.

The reported figures must be read with one limitation in mind. Each configuration was measured in a single end-to-end sweep and was not independently replicated, so the results are single-run per configuration. The within-run coefficient of variation, retained for every timed series, is small on this platform — typically well below one percent on the headline speedup curves and never above roughly two percent — which indicates that individual data points are stable; it does not, however, substitute for between-run replication. The content-skew diagnostic of the next chapter follows a stricter protocol on this point: each of its configurations is executed as five independent process invocations, each with one warm-up and three timed iterations, and the reported value is the median across invocations. A replication pilot run under the same protocol on the secondary processor, over the canonical corpus, illustrates why the distinction matters: up to eight threads the five invocations agree closely, but at that machine's full logical thread count the between-run dispersion grows large even though each individual run remains internally stable — a caution against reading any single invocation at a saturated operating point as definitive. Independent replication of the key configurations on the canonical workstation, reported as a median and interquartile range, is left as a strengthening of the present single-run figures. The desktop platform runs under the `performance` governor at a sustained all-core clock, so the published values are steady-state rather than short-burst figures.

## 5.7 CORRECTNESS VALIDATION

Correctness is a precondition for any performance claim: a faster scan is worthless if it does not return exactly the matches that the sequential algorithm returns. The goal of every optimization here is to reduce execution time while preserving the output of the

sequential baseline, so each parallel variant was required to reproduce that output before it was benchmarked. Correctness was established at three complementary levels, described in turn below.

First, at the level of the match *set*, the automated test suite requires every strategy to reproduce the exact set of matches of the sequential baseline — each match identified by its end position and pattern identifier — across thread counts of $\{1, 2, 3, 4, 7, 8\}$. The same suite verifies that the level-synchronous parallel construction yields an automaton bit-for-bit identical to the sequentially built one, so that swapping in the parallel build can never alter the reported matches.

Second, at the level of the full-scale *multiset*, the MD5 digest of the canonicalized match stream — the complete set of matches placed in a deterministic order by end position and pattern identifier — was computed for each parallel strategy on the production-scale workloads and compared against the digest of the sequential baseline. Agreement certifies the exact multiset of matches (their count, end positions, and pattern identifiers), a far stronger guarantee than match-count parity; it held for every strategy up to a thread count of $64$, a $2\times$ oversubscription of the thirty-two hardware threads, demonstrating that the partitioning logic is robust well beyond the physical thread count.

Third, at the level of segment boundaries and emission order, the strategies whose thread-local results are merged in text order (the static-chunking strategies) reproduce even the digest of the raw, un-canonicalized emission stream, confirming that every boundary-straddling match is assigned to exactly one segment. The dynamic-dispatch and dictionary-sharded strategies emit the identical multiset in a different order, so for them only the canonicalized digest coincides, which is the expected and correct behaviour.

Finally, ThreadSanitizer reported no data races during the parallel search, empirically confirming the read-only-automaton invariant and the absence of unsynchronized shared writes on the hot path.

## 5.8 EXPERIMENTAL DESIGN AND HYPOTHESES

The objective of this work is to accelerate multi-pattern matching by parallelizing Aho–Corasick on shared-memory multi-core CPUs. That objective is decomposed into a set of explicit hypotheses, each phrased so that an experiment can falsify it and each isolating a single design axis while the others are held fixed. Table 6 maps every hypothesis to the experiment that tests it and to the place in the next chapter where its outcome is reported; throughout, the comparison is against the sequential baseline for headline speedup and against the strongest text-parallel strategy when the question concerns composition.

Each row answers a different question: how performance scales with threads, what

Table 6: Hypotheses, the experiment that tests each one, and where the outcome is reported.

| Hypothesis | Experiment | Reported in |
|---|---|---|
| Text chunking scales sublinearly on a multi-core CPU but suffers load imbalance when segments are sized uniformly. | Speedup curves (Phase A) | Fig. 13 |
| The automaton footprint, not the corpus content, governs throughput once the working set escapes the cache. | Footprint and cross-corpus sweeps (Phases B–C) | Table 7 |
| The flattened output layout reduces emission cost in both the sequential and the parallel case. | Layout comparison | Table 8 |
| On homogeneous cores, dynamic dispatch balances the workers at least as well as static chunking, while static frequency weighting adds overhead rather than benefit. | Per-thread diagnostic (Phase D) | Table 9 |
| Coarser task granularity improves the dynamic scheduler by amortizing atomic-counter contention. | Task-granularity sweep (Phase G) | Table 10 |
| A level-synchronous parallel construction reduces build time for large dictionaries. | Build sweep (Phase E) | Table 11 |
| Prefix-based dictionary sharding recovers throughput in the memory-bound regime by shrinking the per-shard working set. | Sharding experiment | Table 12 |
| A two-dimensional composition of sharding and chunking combines cache locality with text parallelism. | Composition experiment | Table 13 |
| When matches concentrate in one region of the input, static partitions lose throughput to barrier idle, while dynamic dispatch redistributes the work. | Content-skew diagnostic | Table 15 |

governs throughput, whether the emission layout pays off, how the workers stay balanced and how task granularity tunes the dynamic scheduler, whether construction parallelizes, whether dictionary parallelism helps on its own or in composition, and whether the dynamic scheduler's balancing advantage materializes when the input content is skewed. Several hypotheses anticipate negative or qualified outcomes, and reporting a strategy that fails to outperform a simpler counterpart is treated as a result in its own right, since it delimits where plausible optimizations stop paying off on this class of hardware. The next chapter presents the outcome of this plan, following the same order.


6

# 6 RESULTS AND DISCUSSION

This chapter presents the experimental evaluation of the proposed parallel Aho-Corasick implementation. Following the methodology described in the previous chapter, each results section follows the same reporting sequence: a brief introduction states the experimental question and configuration, the corresponding figure or table is presented before the discussion, the measured pattern is then described without interpretation, the explanation is separated from the observation, and the subsection closes with a local conclusion tied to the corresponding hypothesis. This ordering keeps what was measured separate from what is inferred.

The chapter is organized as follows. It first characterizes how the execution platform responds to a growing automaton footprint, establishing the experimental context for the analyses that follow. The subsequent sections then evaluate the individual optimization axes introduced earlier—text partitioning and its scalability, the flattened output layout, load balance on uniform cores, task granularity, parallel construction, dictionary sharding, and the two-dimensional composition of sharding with chunking—each as a controlled experiment that varies a single axis. Two short, clearly delimited diagnostics then isolate effects that the canonical, uniform setting cannot exhibit: load imbalance across cores with different performance classes, and load imbalance induced by input content whose matches concentrate in one region of the text. The final section consolidates the findings and positions them against the related literature.

All search and construction figures reported here come from the same sustained measurement campaign on the AMD Ryzen 9 9950X workstation, and the end-to-end pipeline times are reconstructed from that same campaign; the sole exceptions are the two closing diagnostic comparisons—load imbalance across heterogeneous cores and load imbalance induced by skewed content—explicitly identified where they appear, which are the only places a second processor is used. Every parallel strategy was first verified to reproduce the exact match set of the sequential baseline. Throughout, speedup is measured against the single-threaded sequential baseline, and the reported values are the sustained, under-load figures obtained at the `performance` governor's steady all-core clock.

## 6.1 IMPACT OF THE AUTOMATON FOOTPRINT ON THROUGHPUT

Before evaluating the proposed optimizations, this experiment characterizes how the execution platform responds to increasing automaton footprints. The question it answers is whether single-threaded scanning throughput is governed by the size of the automaton or by the content of the text. To isolate the footprint, the dictionary is swept from one hundred patterns to roughly forty-five thousand while the corpus is held fixed and a single thread is used, which removes any threading confound from the measurement. Table 7 reports the resulting throughput.

Table 7: Single-threaded scanning throughput as the automaton footprint grows (corpus fixed, single thread).

| Dictionary | Patterns | Automaton | Throughput (MB/s) |
|---|---|---|---|
| Snort-100 | 100 | 1.93 MiB | 629.0 |
| Snort-1k | ∼1,000 | 11.92 MiB | 452.3 |
| Snort (full) | 4,188 | 55.23 MiB | 328.9 |
| Emerging Threats | 44,678 | 506.78 MiB | 210.2 |

Corpus fixed; single thread.

Throughput falls monotonically as the automaton grows, from $629.0$ MB/s at $1.93$ MiB (Snort-100) to $210.2$ MB/s at $506.78$ MiB (Emerging Threats)—a $66.6$% corpus is identical in every row, the degradation tracks the automaton size alone, and not the text content.

A plausible explanation for this behaviour is the memory hierarchy of the platform. Each input byte triggers a lookup in the state-transition table, and the $64$ MiB last-level cache is split into two $32$ MiB slices, one per core complex. The full Snort automaton ($55$ MiB) already exceeds a single $32$ MiB slice, and the largest dictionary produces an automaton about eight times the aggregate $64$ MiB cache, so an increasing share of transitions is likely to miss cache and reach main memory as the automaton grows. This attribution is an inference: memory bandwidth, cache-hit and cache-miss counters were not instrumented, and the magnitude of the effect is specific to this platform's cache size rather than a property of the algorithm itself. Figure 11 illustrates the two regimes schematically.

The local conclusion is that the automaton footprint—not the text content—is the dominant factor in single-threaded throughput. The same degradation also persists under parallel execution: at the full thread count, the speedup of the best strategy (against the sequential baseline) falls from roughly $25.8\times$ for Snort-100 to $19.2\times$ for Emerging Threats, and the absolute throughput collapses from $16{,}230$ MB/s to about $4{,}046$ MB/s—consistent with the memory subsystem rather than the thread count being

the binding factor (Figure 12). This is revisited in the scaling analysis below, and it frames the interpretation of every subsequent result.

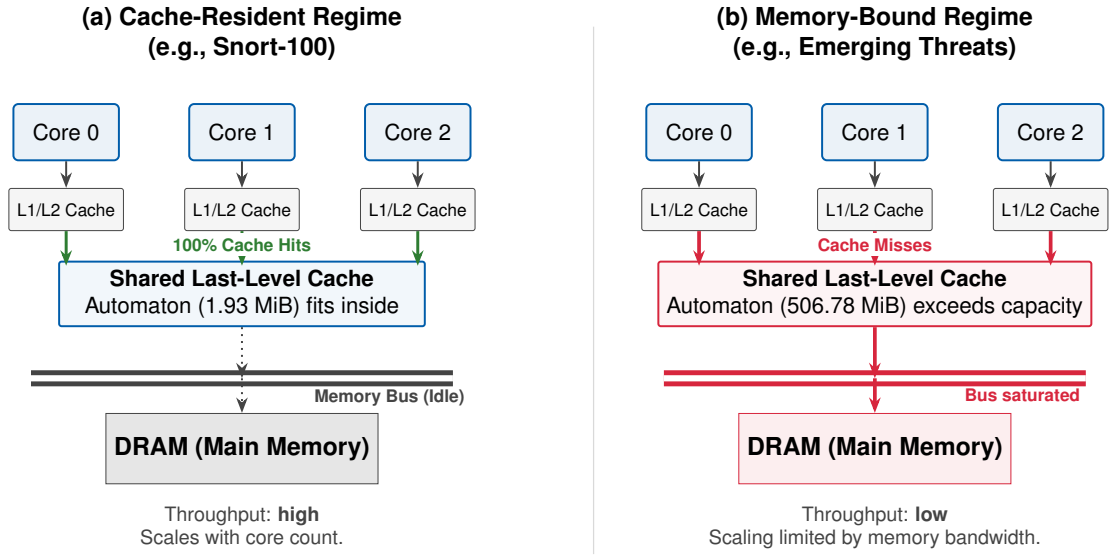



Figure 11: Cache-residency regimes governing throughput: (a) a small automaton fits within the shared LLC, so transitions are likely to be served from cache; (b) a memory-bound automaton ($\gg$LLC) is likely to produce more cache misses and place sustained pressure on the memory bus, flattening the scaling curve.

## 6.2 SCALABILITY OF TEXT PARALLELISM

This experiment evaluates the primary parallelization strategy—partitioning the input text across worker threads—by measuring how search speedup scales with the number of worker threads. It tests the hypothesis, stated in the methodology, that text chunking scales sublinearly on a multi-core CPU, and it does so for the two footprint regimes identified above: a moderate dictionary comparable in size to the cache (Snort, 55 MiB) and a memory-bound dictionary far larger than it (Emerging Threats, 507 MiB). The curves are measured over the canonical Enron corpus across the full thread sweep; the eight-fold replica ($\sim$10.6 GiB, `enron_x8`) reproduces them, confirming that the scaling is not an artifact of a small corpus.

The thread count is swept up to thirty-two. This is not an arbitrary cut-off: the workstation has sixteen physical cores with two-way simultaneous multithreading (SMT), so $T = 32$ is the full hardware concurrency of the platform and larger values would merely oversubscribe the cores. The sweep therefore covers the platform's entire thread budget, separating the sixteen-thread point (one worker per physical core) from the thirty-two-thread point (both SMT siblings active). Figure 13 shows the resulting speedup curves.

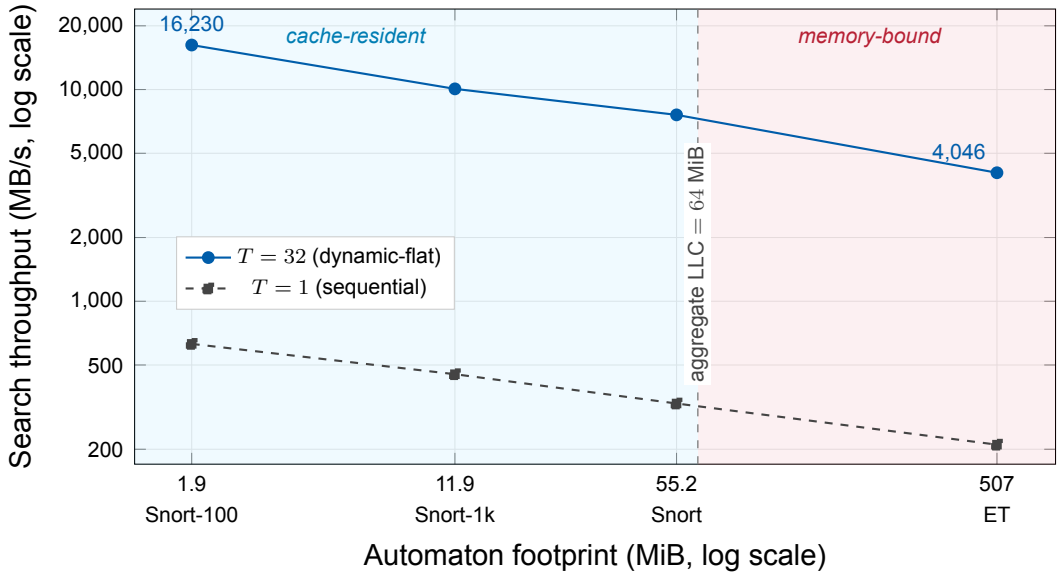



Figure 12: Search throughput as a function of the automaton's memory footprint, on the canonical workstation, at both ends of the thread budget. Each point is one dictionary—Snort-100, Snort-1k, full Snort, and Emerging Threats—so the corpus is held fixed and only the automaton size varies, from $1.9$ to $507$ MiB. Throughput falls monotonically on both curves as the automaton grows: the single thread drops from $629$ to $210$ MB/s, and the thirty-two-thread champion (`pthread_dynamic_flat`) collapses from $16{,}230$ to $4{,}046$ MB/s. The dashed line marks the aggregate $64$ MiB last-level cache (two $32$ MiB slices); the full Snort automaton ($55$ MiB) already exceeds a single slice, and Emerging Threats ($507$ MiB) is about eight times the whole cache, so a growing share of state transitions must be served from main memory. Both axes are logarithmic.

The two regimes behave differently, but both scale monotonically to the full thread count. With the moderate dictionary, the winning strategy (`pthread_dynamic_flat`) rises steadily to $22.91\times$ at thirty-two threads ($7{,}542$ MB/s; CV $0.2$% intrusion-detection workload. With the memory-bound dictionary the same strategy reaches $18.96\times$ ($3{,}979$ MB/s); the eight-fold replica confirms the Snort result ($22.64\times$) and produces only a technical tie in the memory-bound case: `pthread_chunked_flat` reaches $19.83\times$, while `pthread_dynamic_flat` reaches $19.79\times$. Both curves continue increasing through the final measured point: the peak is at $T = 32$ in the canonical scenarios. Both regimes scale sublinearly, the moderate one more steeply than the memory-bound one.

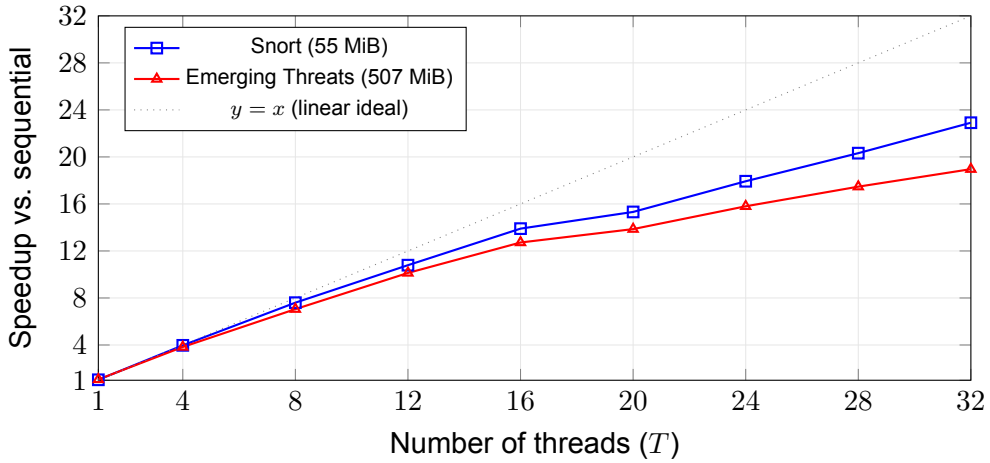



Figure 13: Speedup of text parallelism versus thread count for the winning strategy (`pthread_dynamic_flat`) on the canonical Enron corpus, for a moderate dictionary (Snort, 55 MiB) and a memory-bound dictionary (Emerging Threats, 507 MiB). Scaling is monotonic to the full thirty-two hardware threads; the sixteen-thread mark separates the physical cores from the SMT siblings. The per-point coefficient of variation stays below about $2\%$ bars would fall within the markers and are omitted.

The gap between the two curves is the central observation of this experiment. Its most likely cause is contention for the shared memory bus: once the automaton is about eight times the last-level cache, each added thread competes for a memory subsystem that is already the binding resource, so the memory-bound dictionary gains less per thread than the cache-resident one. This is consistent with the memory bus, rather than the core count, limiting the wider regime, and it remains an inference drawn from the shape of the scaling curves and their agreement with the footprint experiment, not a direct measurement: memory bandwidth and cache counters were not instrumented, a limitation revisited in the discussion of threats to validity.

Parallel efficiency (Eq. 4) makes the shape of the scaling concrete. One definitional care is needed first: the winning strategy inherits the flat emission layout, so dividing its headline speedup by the thread count would credit the layout gain—quantified later in this chapter (Table 8)—to parallelism and report efficiencies above $100\%$ at trivial thread counts. Efficiency is therefore computed against the flat sequential baseline (`sequential_flat`), the strongest single-threaded implementation in the laboratory, so that the curve isolates the contribution of parallelism itself. Figure 14 plots the result over the full thread sweep, exposing the saturation that the rising speedup curves of Figure 13 compress. From near-perfect efficiency at one thread, the moderate regime falls to $83.5\%$ physical core—and the memory-bound regime to $74.3\%$ SMT sibling on each core raises throughput by a further $1.65\times$ (moderate) and $1.49\times$ (memory-bound) but lowers efficiency to $68.8\%$ at thirty-two threads. The loss is not evenly distributed: crossing from sixteen to eighteen threads—the first configuration in which SMT siblings

share a physical core—costs about eight percentage points in a single step in both regimes ($83.5\%$ anywhere on either curve. That drop is consistent with SMT sharing, which can hide memory latency without doubling compute, and with residual memory contention in the wider regime. It is therefore treated as an inference from the curve and the footprint result, not as a measured sequential bottleneck or a counter-level measurement.

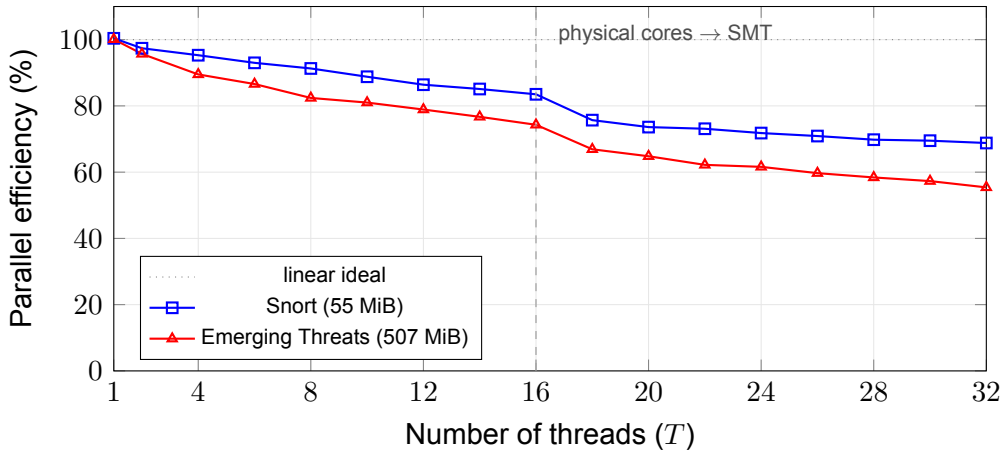



Figure 14: Parallel efficiency—speedup divided by thread count (Eq. 4)—for the winning strategy (`pthread_dynamic_flat`) on the canonical Enron corpus, computed against the flat sequential baseline (`sequential_flat`) so that the emission-layout gain is not credited to parallelism. While the speedup itself rises monotonically to $T = 32$ (Figure 13), the efficiency view exposes the saturation of the parallel gain: each added thread contributes less, and the largest single loss occurs when the sweep crosses the dashed line at sixteen threads and the first SMT siblings begin to share physical cores. The gap between the two regimes widens with the thread count, reaching about thirteen percentage points at $T = 32$, consistent with the memory-bandwidth contention discussed above.

The local conclusion is that text parallelism delivers a sublinear but substantial speedup whose ceiling is set by the footprint regime: it reaches $22.91\times$ where the working set is moderate and $18.96\times$ once the automaton is much larger than the cache. This confirms the sublinear-scaling hypothesis and sharpens it, by tying the residual gap to the automaton footprint rather than to the number of cores.

A separate experiment fixed the dictionary and strategy while changing the corpus, to confirm that the text content is not the main factor. Holding `pthread_dynamic_flat` and the full Snort dictionary fixed, the sequential throughput on the natural-language SimpleWiki corpus ($342.2$ MB/s) slightly exceeds that on Enron ($329.4$ MB/s), and at thirty-two threads the ordering is the same ($7{,}976$ MB/s versus $7{,}595$ MB/s), both with a coefficient of variation below $0.3\%$ *faster*, not slower, than the e-mail corpus. Because Aho-Corasick performs one transition per input byte regardless of match density, the

footprint of the automaton dominates the behaviour far more than the characteristics of the corpus.

## 6.3 THE FLATTENED OUTPUT LAYOUT

The proposed output-layout optimization replaces the pointer-chasing emission of matches with a sequential read of a contiguous array. Because it changes only the match-output representation, its effect is measured first on a single thread while the automaton footprint is varied. The question is whether removing one layer of indirection from match emission produces a visible gain when the transition table and output metadata grow from a small to a large working set. Table 8 reports the measured gain by automaton footprint.

Table 8: Single-threaded gain of the flattened output layout, by automaton footprint.

| Dictionary | Automaton | Baseline (MB/s) | Flattened (MB/s) | Gain |
|---|---|---|---|---|
| Snort-100 | 1.93 MiB | 629.0 | 647.8 | $+3.0\%$ |
| Snort (full) | 55.23 MiB | 328.9 | 342.6 | $+4.2\%$ |
| Emerging Threats | 506.78 MiB | 210.2 | 224.6 | $+6.9\%$ |

Single thread.

The measured gain is positive in every row and grows with the automaton footprint: $+3.0\%$ and $+6.9\%$ as the automaton grows, but the flattened representation consistently improves the same searcher under the same dictionary and corpus.

The interpretation is that the output layout removes work from a part of the scan that becomes increasingly expensive as the working set grows. The optimization does not change the transition function and therefore cannot remove the main per-byte cost of the algorithm; it only turns match emission from a pointer walk into a linear access pattern. This explains both why the single-threaded gain remains modest and why it is largest for the largest automaton. The cache hierarchy of the workstation provides supporting evidence for this reading, as shown in the footprint experiment, but the argument does not depend on a specific last-level-cache size: a processor with a different cache organization would move the boundary between regimes, not change the fact that a larger automaton creates a larger working set.

In the parallel case, the layout is inherited by the chunked strategies, and the flattened variants lead their pointer-chasing counterparts in both intrusion-detection-scale regimes. The winning strategies at thirty-two threads—`pthread_dynamic_flat` at $22.91\times$ for the moderate dictionary and $18.96\times$ for the large dictionary—both incorporate this layout. The local conclusion is therefore limited but useful: flattening is not the main source of parallel speedup, but it is a cheap improvement that becomes

more valuable as the automaton footprint grows.

## 6.4 LOAD BALANCE AND RUN-TO-RUN STABILITY

This experiment asks how the work-distribution and emission strategies compare on uniform cores, where every core runs at the same clock and a uniform text partition should already be balanced. It uses the canonical Snort workload at the full thread count and compares, for each strategy, the search throughput and the coefficient of variation across the timed iterations of the same run. The partitioning policies are defined in the proposal chapter (Figure 7). Table 9 reports both quantities.

On these uniform cores every strategy is already well balanced: the coefficient of variation stays below one percent for all of them, so there are no stragglers for the balancing policies to remove. The ranking is therefore decided by raw throughput rather than by stability. Dynamic dispatch with the flattened layout leads at $7{,}542$ MB/s; the flattened layout alone adds a few percent over pointer-chasing emission ($7{,}398$ against $6{,}920$ MB/s for static chunking), and dynamic dispatch adds a further increment by smoothing the residual imbalance across tasks. The frequency-weighted, topology-aware variant does *not* lead: paired with the flat layout it reaches $7{,}302$ MB/s, below the $7{,}398$ MB/s of plain flat chunking, so its extra affinity and per-core weighting logic is at best neutral and slightly counterproductive on uniform hardware, where the weights it computes are all equal. Per-worker timings confirm the picture: at thirty-two threads the dynamic–flat workers finish within about $9\%$ spread attributable to memory-system jitter rather than to a structural imbalance. Whether the topology-aware machinery ever pays off is a question only a heterogeneous processor can answer; that comparison — the single place the Intel Core i5-1235U enters this work — is presented separately in Section 6.9 on load imbalance across heterogeneous cores.

Table 9: Throughput and iteration-to-iteration stability of the text-parallel strategies (thirty-two threads, Snort, canonical corpus).

| Strategy | Throughput (MB/s) | Variation (CV) |
|---|---|---|
| Static homogeneous chunking | 6,920.4 | $0.53\%$ |
| Static chunking (cache-padded) | 7,099.3 | $0.49\%$ |
| Topology-aware (frequency-weighted) | 7,065.7 | $0.75\%$ |
| Topology-aware with flattened output | 7,302.3 | $0.49\%$ |
| Dynamic dispatch | 7,236.4 | $0.17\%$ |
| Flattened output with chunking | 7,398.4 | $0.20\%$ |
| Dynamic dispatch with flattened output | 7,542.3 | $0.24\%$ |

Canonical workload, thirty-two threads; the coefficient of variation is measured across the five timed iterations of the same run.

## 6.5 TASK GRANULARITY OF THE DYNAMIC SCHEDULER

The dynamic scheduler pulls text tasks from a shared atomic counter, so its performance depends on how large each task is: tasks that are too small multiply contention on the counter, while tasks that are too large risk leaving a worker without work near the end. This experiment sweeps the number of text tokens per task for the winning strategy at the full thread count, holding the dictionary and corpus fixed. Table 10 reports the result.

Table 10: Effect of task granularity on the dynamic scheduler (`pthread_dynamic_flat`, Snort, canonical corpus, thirty-two threads).

| Tokens per task | Throughput (MB/s) | Variation (CV) |
|---|---|---|
| 1 | 7,413.9 | 0.33% |
| 4 | 7,600.9 | 0.17% |
| 16 | 7,663.8 | 0.15% |
| 64 | 7,756.0 | 0.14% |
| 256 | 7,804.1 | 0.04% |

Headline sweep, canonical workload, thirty-two threads.

Throughput increases monotonically with task size, from 7,414 MB/s at one token per task to 7,804 MB/s at two hundred and fifty-six—about a 5% —while the coefficient of variation stays below 0.35% is that, on this workload, coarser tasks amortize the atomic-counter contention of the bag-of-tasks scheme without harming load balance: the corpus is close to uniform, so large tasks do not create end-of-scan stragglers. The local conclusion is that the dynamic scheduler is best run with coarse granularity here; a finer granularity would only become necessary on a skewed corpus, where match density varies enough across the text to make small tasks worthwhile—a case this uniform corpus does not exercise. The content-skew diagnostic of Section 6.10 creates exactly that condition, although it evaluates the scheduling policies at their default granularity rather than re-sweeping the task size.

## 6.6 PARALLEL CONSTRUCTION OF THE AUTOMATON

This experiment evaluates whether parallel construction reduces operational rule-reloading latency. The implemented variant parallelizes the breadth-first traversal that computes failure links, because each level depends only on shallower levels and can therefore be processed in parallel with a barrier between levels. The trie insertion remains sequential; only the traversal is parallelized. The level-synchronous construction strategy and its dependency structure are defined in the proposal chapter (Figure 9). Table 11 reports the build speedup across thread counts, rather than a single peak, so that the full thread-count behaviour is visible.

The thread-count breakdown shows that parallel construction only pays off for the large dictionary, where it peaks at $1.59\times$ at sixteen threads (reducing the build from $324$ to roughly $203$ milliseconds) before easing to $1.47\times$ at thirty-two. The moderate dictionary barely benefits, peaking at $1.17\times$ at four threads and regressing to $0.57\times$ at thirty-two, while the smallest dictionary is slower than sequential at every thread count, because the per-level spawn-and-join overhead exceeds the work. This optimization should therefore be understood as an operational benefit—a reduction in rule-reloading latency for systems that frequently update their signatures—rather than as a contribution to scanning throughput; its share of the end-to-end pipeline is quantified in the brief consistency check below.

Table 11: Parallel automaton construction: build speedup versus the sequential build, by thread count.

| Dictionary | States | Seq. (ms) | $T{=}2$ | $T{=}4$ | $T{=}8$ | $T{=}16$ | $T{=}32$ |
|---|---|---|---|---|---|---|---|
| Snort-100 | 1,939 | 0.8 | $0.70\times$ | $0.70\times$ | $0.52\times$ | $0.35\times$ | $0.21\times$ |
| Snort (full) | 55,479 | 28.3 | $0.98\times$ | $1.17\times$ | $1.02\times$ | $0.85\times$ | $0.57\times$ |
| Emerging Threats | 508,896 | 323.9 | $1.13\times$ | $1.38\times$ | $1.58\times$ | $1.59\times$ | $1.47\times$ |

Build speedup is the sequential build time divided by the parallel build time at each thread count; values below $1.0\times$ mean the parallel build is slower than the sequential one.

## 6.7 PARTITIONING THE DICTIONARY: A QUALIFIED NEGATIVE RESULT

Partitioning the dictionary, rather than the text, was evaluated as an alternative, assigning patterns to shards by their first byte (prefix sharding). The searcher scales the number of shards with the thread count, so Table 12 reports throughput across shard counts and compares the result against plain text parallelism on the canonical workload.

Prefix sharding peaks at only $1.89\times$ over the baseline at eight shards and then declines as more shards are added, ending at $1.78\times$ at thirty-two. That it peaks and reverses so far below the thread budget—rather than continuing to scale like text chunking—indicates that the benefit comes from improved cache locality within each smaller sub-automaton rather than from added parallelism; past the peak, coordination and duplicated text traffic dominate. Even at its best it attains only about $8$% ($620.9$ against $7{,}542.3$ MB/s). The conclusion is that, on this class of hardware, partitioning the text dominates partitioning the dictionary: a real but small effect, not a competitive strategy.

Table 12: Dictionary sharding (by prefix) across shard counts, compared to text parallelism (Snort, canonical corpus).

| Configuration | Throughput (MB/s) | vs. sequential |
|---|---|---|
| Prefix sharding, 2 shards | 481.1 | $1.46\times$ |
| Prefix sharding, 4 shards | 570.1 | $1.73\times$ |
| Prefix sharding, 8 shards (peak) | 620.9 | $1.89\times$ |
| Prefix sharding, 16 shards | 605.0 | $1.84\times$ |
| Prefix sharding, 32 shards | 586.0 | $1.78\times$ |
| Text chunking, dynamic flat (32 threads) | 7,542.3 | $22.91\times$ |

Speedup is relative to the single-threaded sequential baseline ($329.2$ MB/s); text chunking is shown at thirty-two threads for reference.

## 6.8 COMBINED STRATEGY EVALUATION

A natural hypothesis is that combining dictionary sharding (for cache locality) with text chunking (for parallelism) would recover the throughput lost in the memory-bound regime, by making each sub-automaton small enough to approach the cache. Table 13 tests this two-dimensional composition against the canonical dynamic-flat text-chunking strategy and disproves the hypothesis.

The composition is slower than text chunking in every measured scenario, by $19$% limiting resource: each shard must scan the entire input text, so $K$ shards multiply the

text traffic on the memory bus by a factor of $K$. The extra reads outweigh any locality gained inside the smaller sub-automata. Shrinking the largest sub-automaton below the $64$ MiB last-level cache would take about eight shards, each rescanning the full corpus, so the eightfold text traffic is not recoverable on a memory bus of this bandwidth. This negative result is an important contribution: it closes a plausible region of the design space and reinforces the conclusion that text parallelism is the correct axis on memory-bound workloads. This difference in memory traffic is illustrated in the proposal chapter (Figure 10).

Table 13: Two-dimensional composition versus the canonical dynamic-flat text-chunking strategy (thirty-two threads).

| Workload | Composition (MB/s) | Text chunking (MB/s) | Ratio |
|---|---|---|---|
| Snort, canonical corpus | 5,512.9 | 7,542.3 | $0.73\times$ |
| Snort, $\sim$10.6 GiB | 5,513.2 | 7,459.1 | $0.74\times$ |
| Emerging Threats, canonical corpus | 3,214.1 | 3,979.3 | $0.81\times$ |
| Emerging Threats, $\sim$10.6 GiB | 3,311.3 | 4,174.6 | $0.79\times$ |

Thirty-two threads in both columns; the comparator is the `pthread_dynamic_flat` text chunking of Table 12.

As a final end-to-end consistency check, the same canonical sweep reconstructs the complete build-plus-one-scan pipeline on the $\sim$10.6 GiB corpus. The resulting speedups are $22.23\times$ for Snort and $17.71\times$ for Emerging Threats, slightly below the search-only speedups on the same corpus ($22.64\times$ and $19.79\times$) because the parallel column still pays the sequential construction cost. This does not introduce a separate experimental result; it only confirms that construction remains a small, bounded overhead and that the complete pipeline is still dominated by the scan.

## 6.9 LOAD IMBALANCE ON HETEROGENEOUS CORES

Every experiment so far ran on the homogeneous workstation, where identical cores make a uniform text partition balanced by construction. The topology-aware, frequency-weighted policy introduced in the proposal (Figure 7) was designed for a different situation: a processor whose cores are not equally fast. This section isolates that effect on the only such processor available in this work — an Intel Core i5-1235U, a hybrid design that pairs Performance cores with slower Efficiency cores — and contrasts it with the homogeneous workstation. Its sole purpose is to establish where topology-aware balancing earns its complexity; the i5 is not a performance platform and none of the headline figures reported earlier depend on it.

The experiment asks whether the balancing policies actually reduce load imbalance,

and on which machine. It uses the canonical Snort workload with per-thread instrumentation, runs each processor at its full logical thread count — twelve on the i5, thirty-two on the workstation — and reports, for each work-distribution strategy, the coefficient of variation across the timed iterations of the run. Table 14 places the two machines side by side.

Table 14: Iteration-to-iteration coefficient of variation of the work-distribution strategies on a heterogeneous versus a homogeneous processor (per-thread instrumentation, Snort workload, each machine at its full logical thread count).

| **Work-distribution strategy** | **i5-1235U (12T)** | **Ryzen 9 9950X (32T)** |
|---|---|---|
| Static homogeneous chunking | 39.9% | |
| Static chunking, cache-padded | 52.9% | |
| Topology-aware (frequency-weighted) | 1.7% | |
| Dynamic dispatch | 3.3% | |

Derived from the per-thread instrumentation of each processor. The i5-1235U is heterogeneous (two Performance cores plus eight Efficiency cores) and the Ryzen 9 9950X is homogeneous; the coefficient of variation is measured across the timed iterations of the run, at each machine's full logical thread count.

The two machines behave in opposite ways. On the hybrid i5, static chunking is highly unstable from one timed iteration to the next: the coefficient of variation is $39.9\%$ per-worker timings point to the cause: under static chunking the slowest worker takes about $1.70\times$ as long as the fastest, consistent with equal-size segments landing on Efficiency cores that run well below the Performance cores and finishing late at the barrier while the faster threads idle. Because the operating system is free to place the threads on either core type, changing placement across iterations would make the barrier time swing; this interpretation fits the large variation, though no scheduler-placement trace was recorded to confirm it directly. The two policies designed for this case remove most of the imbalance: the topology-aware, frequency-weighted strategy, which pins each worker and sizes its segment to the measured core speed, cuts the variation to $1.7\%$ worker spread to $1.17\times$; dynamic dispatch, which lets faster cores claim more tasks from a shared counter, reaches $3.3\%$ That two independent mechanisms both collapse a forty-to-fifty-percent variation to a few percent indicates that the instability is a load-imbalance effect rather than measurement noise.

On the homogeneous workstation the same strategies show almost no differentiation. Static chunking already runs at a coefficient of variation of $4.1\%$ cache-padded static variant is at $0.9\%$ $1.13\times$ for the plain static partition. The balancing policies land at $0.6\%$ the cache-padded static variant. With identical cores there are no stragglers to correct, so the frequency weights the topology-aware policy computes are all equal and it reduces to plain static chunking. This mirrors the earlier finding on uniform cores

(Table 9), where the strategy ranking was decided by raw throughput rather than by balance.

The local conclusion is that topology-aware, frequency-weighted partitioning is a contribution specific to heterogeneous, hybrid-core hardware: it is decisive on the i5, where it converts a wildly variable static partition into a stable one, and inert on the homogeneous workstation, where the winning strategy is instead dynamic dispatch with the flattened output layout. The thesis strategy therefore holds without promoting the i5 to a measurement platform — the workstation supplies every performance figure, while the hybrid processor serves only to show that the load-balancing machinery addresses a real effect where that effect exists.

## 6.10 LOAD IMBALANCE FROM SKEWED CONTENT

The previous section isolated the load imbalance that unequal cores create. This second, closing diagnostic isolates the complementary source of imbalance: the content of the input itself. Even when every core is identical, a static partition leaves workers idle at the final barrier if the matches concentrate in one region of the text, because the workers that own the match-dense segments must emit far more matches than the rest. The canonical corpora are too close to uniform to exercise this effect, so it is measured with the synthetic skew pair described in the methodology (Table 3): two corpora built from the same blocks — hence with identical byte counts and identical total match counts — in which the match-dense blocks are either spread evenly (*uniform*) or concentrated in the first quarter of the file (*clustered*). Because the totals are equal, any difference between the two corpora is attributable to *where* the work sits, not to how much of it there is.

Like the previous section, this experiment runs on the Intel Core i5-1235U at its full logical thread count and is a diagnostic, not a source of headline figures. Each configuration was executed as five independent process invocations, and every value reported below is the median across those invocations. The absolute throughputs are not comparable to the workstation tables: the machine differs, and the synthetic pair carries a higher match density than the canonical Enron corpus. Table 15 reports the median search throughput of each work-distribution strategy on the two corpora.

The pattern in Table 15 is symmetric. Moving from the uniform to the clustered corpus, the static strategies lose between $16$% median throughput, while the dynamic strategies gain between $16$% and overtake every static variant: on the clustered corpus the dynamic flat strategy reaches $456.8$ MB/s against $333.2$ MB/s for the best static one with the Snort dictionary, and $320.8$ against $241.0$ MB/s with Emerging Threats. The per-worker instrumentation explains the reversal. Table 16 reports, for the Snort dictionary, two quantities derived from the per-worker scan times of each invocation: the worker spread, defined as the difference between the slowest and the fastest worker divided

by the mean worker time, and the barrier idle fraction, defined as one minus the ratio of the mean to the maximum worker time — the share of the parallel window in which the average worker has finished and is waiting at the barrier.

Table 15: Median search throughput on the content-skew corpus pair (i5-1235U, twelve threads, medians of five independent invocations).

| | Snort (MB/s) | | | Emerging Threats (MB/s) | | |
|---|---|---|---|---|---|---|
| **Strategy** | **Unif.** | **Clust.** | **Change** | **Unif.** | **Clust.** | **Change** |
| Static chunking (cache-padded) | 446.4 | 317.0 | $-29.0\%$ | | | |
| Static chunking, flattened output | 473.1 | 333.2 | $-29.6\%$ | | | |
| Topology-aware (frequency-weighted) | 394.9 | 316.7 | $-19.8\%$ | | | |
| Dynamic dispatch | 371.6 | 430.8 | $+15.9\%$ | | | |
| Dynamic dispatch, flattened output | 390.6 | 456.8 | $+16.9\%$ | | | |

Both corpora hold bytes and total matches fixed; only the spatial placement of the match-dense blocks differs.

A representative invocation makes the numbers concrete. Under flattened static chunking on the clustered corpus, the three workers that own the first quarter of the text emit about $19.2$ million matches each and take $12.3$ to $13.3$ seconds, while the remaining nine emit about $4.1$ million each and finish in $2.7$ to $3.8$ seconds — the run then waits at the barrier for the three loaded workers. Under flattened dynamic dispatch on the same corpus, all twelve workers finish within $10.95$ to $11.31$ seconds even though their match counts still range from roughly $6.8$ to $9.9$ million: the imbalance moves from the workers' finishing times, where it wastes wall-clock time at the barrier, into their task counts, where it is harmless.

Table 16: Worker balance under content skew (Snort dictionary, i5-1235U, twelve threads; medians of five independent invocations).

| | Worker spread | | Barrier idle | |
|---|---|---|---|---|
| **Strategy** | **Unif.** | **Clust.** | **Unif.** | **Clust.** |
| Static chunking (cache-padded) | $18.0\%$ | | | |
| Static chunking, flattened output | $17.4\%$ | | | |
| Topology-aware (frequency-weighted) | $5.9\%$ | | | |
| Dynamic dispatch | $18.5\%$ | | | |
| Dynamic dispatch, flattened output | $21.0\%$ | | | |

Worker spread: (slowest $-$ fastest) worker time over the mean worker time. Barrier idle: $1-$ mean/maximum worker time.

Two further observations qualify the result. First, the topology-aware policy does not correct content-induced imbalance: its worker spread on the clustered Snort corpus

rises from $5.9\%$ essentially matching plain static chunking. This is expected from its design — the policy sizes segments from a model of core speed, and no hardware model can anticipate where the matches are — and it holds even in the one cell where its clustered median does not fall (Emerging Threats, $+6.8\%$ and idle deteriorate there as well ($8.4\%$ $31.1\%$ ones on this machine ($473.1$ against $390.6$ MB/s for the flattened variants), so dynamic dispatch is not universally faster; what the contrast establishes is robustness, not dominance.

This diagnostic carries the caveats of its platform. Core heterogeneity is present in both arms of the contrast, so the uniform-versus-clustered comparison attributes the *direction* of the change to the content, while the magnitudes remain specific to this hybrid processor; the corpora are synthetic; and the experiment has not been repeated on the homogeneous workstation, a limitation taken up in the threats to validity. Within those limits, the local conclusion is direct: when the work is spatially concentrated in the input, every static partition — including the frequency-weighted one — loses throughput to barrier idle, and dynamic dispatch redistributes the work and overtakes them. This is the regime that motivates the bag-of-tasks design beyond its modest margin on uniform inputs.

## 6.11 DISCUSSION AND POSITIONING

Taken together, the results converge on a consistent reading. The dominant obstacle to accelerating Aho-Corasick at intrusion-detection scale appears to be microarchitectural: once the automaton greatly exceeds the last-level cache, throughput is bounded by the memory subsystem, and the most effective response is to parallelize the text while keeping the per-byte transition cost as low as possible. The canonical Enron scenarios have a resolved winner: dynamic dispatch with the flattened output layout (`pthread_dynamic_flat`). On uniform cores, flat emission makes the per-match cost negligible and the bag-of-tasks dispatch smooths the residual imbalance; the only technical tie is the memory-bound eight-fold replica, where static flat chunking reaches $19.83\times$ and dynamic flat reaches $19.79\times$, so no conclusion relies on that ordering. The two closing diagnostics complete this picture: on uniform inputs the margin of dynamic dispatch over static flat chunking is small, and on a saturated hybrid processor it can even invert, but dynamic dispatch is the only evaluated policy that stays balanced under both sources of load imbalance examined in this work — cores of unequal speed and content whose matches concentrate in one region — whereas frequency-weighted partitioning corrects only the former. Its selection as the recommended strategy therefore rests on robustness combined with leading throughput in the canonical scenarios, not on the uniform-corpus margin alone. The gain from added threads narrows as the dictionary grows, which is consistent with the main bottleneck shifting, as the automaton outgrows the cache, from thread scheduling toward memory bandwidth—the regime in

which a cache-friendly emission path helps the most—an interpretation inferred from the measurements rather than confirmed by hardware-counter instrumentation.

These findings can be positioned against the related literature. Compared to approaches that restructure the automaton to improve cache friendliness, such as the head–body decomposition of the matcher for multi-core intrusion detection [2], the present work attains a comparable order of magnitude of throughput while leaving the classical automaton mathematically intact, which keeps correctness easy to validate against the sequential reference. The failure-less variant of the algorithm designed for massively parallel accelerators [5] is, however, unsuitable for a multi-core CPU in this setting, because its independent per-byte traversals multiply work in a way that the memory bus is unlikely to absorb; this is consistent with the negative result observed here for over-partitioning. Single-threaded vectorized engines that replace the automaton with SIMD literal matching [24] operate on an orthogonal axis, exploiting data parallelism within a thread rather than across threads, and are therefore complementary to, rather than competitive with, the inter-thread parallelism studied here. The finding that layout-level changes to the emission path yield real gains aligns with work on transition-table compression for the same algorithm [23]. Finally, the use of realistic rule sets and large corpora follows the guidance on representative signature-based intrusion-detection benchmarks [6], which is what allows the memory-bound regime to be studied, rather than an artificially cache-friendly one.

# 7

# 7 CONCLUSION

This thesis investigated how far Aho–Corasick multi-pattern matching can be accelerated on a shared-memory multi-core CPU when the input is large and the automaton footprint approaches or exceeds the last-level cache. The empirical campaign was centered on a homogeneous AMD Ryzen 9 9950X workstation, and the results support a direct conclusion: for intrusion-detection-scale pattern sets, the dominant barrier is not the state-transition logic itself, but the memory system that must feed many threads traversing the same large automaton.

Three findings summarize the contribution. First, the automaton footprint is the main regime switch. With the dynamic flat strategy at thirty-two threads, throughput falls from about $16.2$ GB/s on the smallest Snort-derived automaton to about $4.0$ GB/s on the Emerging Threats automaton, whose footprint is roughly $507$ MiB. The drop persists even on a desktop processor with sixteen identical cores and a much larger cache hierarchy, which indicates that the memory-bound regime is structural to the workload. Second, the most effective parallelization axis is text partitioning over a shared, read-only automaton. Dictionary sharding and the two-dimensional composition remain useful negative results: they are correct, but they do not approach the throughput of scanning the input once with multiple workers. Third, output layout matters precisely because the search loop is memory-sensitive. Replacing per-state chained emission with a flat contiguous layout reduces pointer chasing and lets the winning configuration combine dynamic task dispatch with flat emission.

The best searcher in the canonical scenarios is therefore `pthread_dynamic_flat`. On the full Snort dictionary and Enron corpus, it reaches $22.91\times$ speedup and $7{,}542$ MB/s at $T = 32$; on the memory-bound Emerging Threats dictionary it still reaches $18.96\times$ and $3{,}979$ MB/s. The eight-fold Enron replica confirms the same ordering for the moderate dictionary ($22.64\times$), while the largest dictionary is a technical tie among flat-emission strategies ($19.79\times$ for dynamic flat and $19.83\times$ for static flat chunking). The cross-corpus experiment further shows that, once the automaton is fixed, text content is secondary: the dynamic flat strategy reaches $7{,}976$ MB/s on SimpleWiki and $7{,}595$ MB/s on Enron. The task-granularity experiment refines the same result, with the coarsest tested dynamic tasks reaching $7{,}804$ MB/s. Parallel construction improves rule loading only modestly, peaking at $1.59\times$ on the largest

dictionary, so the end-to-end pipeline is still governed by search: $22.23\times$ on Snort and $17.71\times$ on Emerging Threats over the eight-fold corpus.

The specific objectives stated in the introduction were met. The systematic literature review identified the gap addressed by this work: a controlled shared-memory Pthreads evaluation of Aho–Corasick under large automata and realistic corpora. The implementation objective was met by a family of C searchers that combine overlap-correct text partitioning, optional load-balancing policies, flat match emission, and level-synchronous construction, while preserving the exact match set of the sequential baseline. The performance objective was met through speedup, throughput, efficiency, end-to-end, and build measurements against the same sequential implementation. Finally, the scalability objective was met by sweeping thread counts up to the workstation's thirty-two logical hardware threads, automaton footprints from $1.93$ to $506.78$ MiB, multiple corpora, and the main strategy combinations.

The two diagnostic experiments have a narrower role in this conclusion. The comparisons in Sections 6.9 and 6.10 do not constitute a second performance platform; they explain when the load-balancing machinery is useful. On heterogeneous cores, topology-aware or dynamic scheduling collapses the large imbalance created by equal-size chunks. When the matches concentrate in one region of the text, only dynamic dispatch keeps the workers balanced: static partitions — including the frequency-weighted one, whose hardware model cannot anticipate where the matches are — lose throughput to barrier idle, while dynamic dispatch redistributes the work and overtakes them. On the homogeneous workstation with near-uniform corpora, the same mechanisms are largely inert, and the throughput winner is selected by memory behavior rather than by placement or balance. Together the diagnostics show that the recommended strategy owes its position to robustness against both sources of load imbalance — hardware and content — in addition to its leading canonical throughput. This supports the design without changing the central empirical platform.

## 7.1 THREATS TO VALIDITY

The principal limitation is that the canonical data set is a single end-to-end sweep: each configuration was measured once, with multiple timed iterations inside that run, but without independent repetitions from which medians and interquartile ranges could be reported. The low within-run coefficients of variation in the headline curves indicate stable individual measurements, but they do not replace between-run replication. A second limitation is architectural coverage. The workstation is a reliable desktop platform, but it is still a single-socket, single-NUMA-node machine; the results therefore do not describe multi-socket NUMA behavior or processors with substantially different memory systems. Third, the interpretation of cache pressure and DRAM bandwidth is an inference from timings, automaton footprint, and scaling curves. No PMU counter in-

strumentation was collected, so statements about cache misses, bandwidth, or pipeline stalls should be read as explanations consistent with the data rather than as directly measured causes. Finally, the canonical corpora are large and realistic but close to uniform, so the headline figures do not quantify the dynamic scheduler's advantage on uneven inputs. The content-skew diagnostic measures that advantage in a controlled way — holding bytes and total matches fixed while concentrating them spatially — but it ran on the hybrid processor, where core heterogeneity is present in both arms of the contrast: the direction of the effect is therefore well supported, while its magnitude on a homogeneous machine remains unmeasured.

## 7.2 FUTURE WORK

Four directions follow from these limitations. The first is to add hardware performance counters to the benchmark harness, so that cache misses, memory bandwidth, and pipeline behavior can be measured directly rather than inferred from timing. The second is to repeat the canonical workstation configurations across independent invocations and report medians with dispersion — the replication pilot on the secondary processor already indicates that saturated operating points are the ones that need this most. The third is to repeat the content-skew diagnostic on the homogeneous workstation, where core heterogeneity cannot contribute to the contrast, and to extend it with a granularity re-sweep, testing whether finer dynamic tasks pay off when the matches are spatially concentrated. The fourth is to widen the hardware surface to additional homogeneous CPUs, hybrid CPUs, and larger NUMA systems, mapping which parts of the strategy ranking are stable and which ones depend on memory hierarchy or core topology.

# REFERENCES

## References


[1] P. Pacheco, *An Introduction to Parallel Programming*, 2nd ed. Morgan Kaufmann, 2011.

[2] C.-L. Lee and T.-H. Yang, "A flexible pattern-matching algorithm for network intrusion detection systems using multi-core processors," *Algorithms*, vol. 10, no. 2, p. 58, 2017.

[3] VirusTotal, "YARA: The pattern matching swiss knife," 2025. [Online]. Available: https://virustotal.github.io/yara/

[4] A. V. Aho and M. J. Corasick, "Efficient string matching: An aid to bibliographic search," *Communications of the ACM*, vol. 18, no. 6, pp. 333–343, 1975.

[5] V. Thambawita, R. G. Ragel, and D. Elkaduwe, "An optimized Parallel Failure-less Aho-Corasick algorithm for DNA sequence matching," *CoRR*, vol. abs/1811.10498, 2018.

[6] M. Aldwairi, M. A. Alshboul, and A. Seyam, "Characterizing realistic signature-based intrusion detection benchmarks," in *Proceedings of the 6th International Conference on Information Technology (ICIT 2018)*. Association for Computing Machinery, 2018.

[7] E. Sitaridi, O. Polychroniou, and K. A. Ross, "SIMD-accelerated regular expression matching," in *Proceedings of the 12th International Workshop on Data Management on New Hardware (DaMoN '16)*. Association for Computing Machinery, 2016.

[8] D. R. Butenhof, *Programming with POSIX Threads*. Addison-Wesley, 1997.

[9] M. J. Flynn, "Very high-speed computing systems," *Proceedings of the IEEE*, vol. 54, no. 12, pp. 1901–1909, 1966.

[10] A. Grama, A. Gupta, G. Karypis, and V. Kumar, *Introduction to Parallel Computing*, 2nd ed. Addison-Wesley, 2003.

[11] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference (AFIPS '67)*, 1967, pp. 483–485.

[12] J. L. Gustafson, "Reevaluating Amdahl's law," *Communications of the ACM*, vol. 31, no. 5, pp. 532–533, 1988.

[13] E. Fredkin, “Trie memory,” *Communications of the ACM*, vol. 3, no. 9, pp. 490–499, 1960.

[14] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Addison-Wesley, 1998.

[15] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Addison-Wesley Professional, 2011.

[16] D. Gusfield, *Algorithms on Strings, Trees, and Sequences: Computer Science and Computational Biology*. Cambridge University Press, 1997.

[17] D. E. Knuth, J. H. Morris Jr., and V. R. Pratt, “Fast pattern matching in strings,” *SIAM Journal on Computing*, vol. 6, no. 2, pp. 323–350, 1977.

[18] R. S. Boyer and J. S. Moore, “A fast string searching algorithm,” *Communications of the ACM*, vol. 20, no. 10, pp. 762–772, 1977.

[19] J. Qu, G. Zhang, Z. Fang, and J. Liu, “A parallel algorithm of string matching based on Message Passing Interface for multicore processors,” *International Journal of Hybrid Information Technology*, vol. 9, no. 3, pp. 31–38, 2016.

[20] D. Deyannis, E. Papadogiannaki, G. Chrysos, K. Georgopoulos, and S. Ioannidis, “The diversification and enhancement of an IDS scheme for the cybersecurity needs of modern supply chains,” *Electronics*, vol. 11, no. 13, p. 1944, 2022.

[21] T. Jepsen, D. Alvarez, N. Foster, C. Kim, J. Lee, M. Moshref, and R. Soulé, “Fast string searching on PISA,” in *Proceedings of the 2019 ACM Symposium on SDN Research (SOSR ’19)*. Association for Computing Machinery, 2019.

[22] D. Tovarňák, “Practical multi-pattern matching approach for fast and scalable log abstraction,” in *Proceedings of the 11th International Joint Conference on Software Technologies (ICSOFT-EA 2016)*. SCITEPRESS, 2016, pp. 319–329.

[23] D. Regéciová, D. Kolář, and M. Milkovič, “Pattern matching in YARA: Improved Aho-Corasick algorithm,” *IEEE Access*, vol. 9, pp. 62 857–62 866, 2021.

[24] H. Xu, H. Chang, W. Zhu, Y. Hong, G. Langdale, K. Qiu, and J. Zhao, “Harry: A scalable SIMD-based multi-literal pattern matching engine for deep packet inspection,” in *IEEE INFOCOM 2023 - IEEE Conference on Computer Communications*. IEEE, 2023.

[25] B. Nichols, J. Buttlar, and J. P. Farrell, *Pthreads Programming: A POSIX Standard for Better Multiprocessing*. O’Reilly & Associates, 1996.

# APPENDICES

# Appendix A - Database Query Protocol

This appendix documents the protocol used to query the raw measurement database from which every result reported in this thesis is derived, including the end-to-end wall-clock consistency check discussed in the Results chapter. Those end-to-end figures are not a separate campaign: each is the sum of two quantities that the canonical sweep already records for the corresponding execution—the construction time (`build_ms`) and a single full-corpus search pass (`mean_ms`). They can therefore be reproduced and audited from the same database as every other figure. The benchmark campaign stores its measurements in a single typed SQLite database located at `parallel-aho-corasick/runs/workstation_2026-06-30/sweep.db`, containing one `runs` table (one row per timed execution) and a set of aggregating views. The headline figures are retrieved from these views instead of the raw execution logs.

## 9.1 ARTIFACT AVAILABILITY

The complete artifacts of this work are publicly available in two Git repositories: the LaTeX source of this document at `https://github.com/MTheus22/acceleration-of-pattern-matching-algorithms-using-parallel-programming`, and the C implementation—together with the searchers, the benchmark harness, the dataset descriptors, and the measurement database—at `https://github.com/MTheus22/parallel-aho-corasick`. Every empirical figure reported in this thesis is derived from the latter repository. After cloning it, the sequential and parallel searchers can be built and exercised through the project Makefile: `make` compiles the laboratory, `make test` checks that every strategy reproduces the exact match set of the sequential baseline, `make digest` runs the MD5 multiset validation described in Section 5.7 (up to sixty-four threads), and `make bench` executes the measurement sweep. The remainder of this appendix documents the schema and the canonical queries of the resulting measurement database, so that each headline number can be traced back to the raw measurements.

## 9.2 DATABASE SCHEMA

Each row of the `runs` table identifies an execution by its phase, pattern dictionary, corpus, searcher strategy, and thread count. It records the timing statistics (minimum, mean, median, and maximum wall-clock time in milliseconds, along with the coefficient of variation across the timed iterations), the derived throughput (in MB/s and Gbit/s), the input volume and match count, and the automaton descriptors (pattern count, number of states, footprint in kibibytes, and sequential or parallel build time). The views listed in Table 17 pre-aggregate the data so no headline figure requires ad hoc post-processing of the raw rows.

Table 17: Aggregating views of the measurement database.

| View | Question it answers |
|---|---|
| `v_speedup` | Speedup of each searcher against the single-threaded sequential baseline. |
| `v_self_speedup` | Self-relative scalability: throughput at $T$ threads over throughput at one thread. |
| `v_footprint` | Throughput as a function of the automaton footprint, at one and thirty-two threads. |
| `v_build` | Speedup of the parallel automaton construction over the sequential build. |
| `v_best` | Best-performing searcher for each dictionary, corpus and thread count. |
| `v_correctness` | Number of distinct match counts per dictionary and corpus (a value of one indicates agreement). |

## 9.3 CANONICAL QUERIES

The queries below reproduce the main results of the study. They can be executed against the database using the standard `sqlite3` client.

```
-- Speedup curve of a searcher (Snort dictionary, canonical corpus)
SELECT thr, speedup_vs_seq FROM v_speedup
WHERE patterns='patterns_snort' AND corpus='enron_corpus'
  AND searcher='pthread_dynamic_flat' ORDER BY thr;

-- Peak speedup per scenario (winning searcher and thread count)
SELECT patterns, corpus, searcher, thr, speedup_vs_seq FROM v_speedup w
WHERE speedup_vs_seq = (SELECT MAX(speedup_vs_seq) FROM v_speedup x
  WHERE x.patterns = w.patterns AND x.corpus = w.corpus)
ORDER BY patterns, corpus;

-- Footprint versus throughput at thirty-two threads (cache blowout)
SELECT patterns, automaton_mib, searcher, mbps FROM v_footprint
WHERE thr = 32 ORDER BY automaton_mib;

-- Speedup of the parallel automaton construction
SELECT patterns, build_threads, build_speedup FROM v_build
ORDER BY patterns, build_threads;
```

```
-- End-to-end wall-clock (build + one full-corpus scan) on enron_x8
SELECT patterns, searcher, thr, build_ms, mean_ms,
       build_ms + mean_ms AS e2e_ms
FROM runs
WHERE corpus='enron_x8' AND thr IN (1, 32)
  AND searcher IN ('sequential','pthread_dynamic_flat')
ORDER BY patterns, searcher, thr;
```

Before any timing was reported, the correctness of every parallel strategy was verified using the `v_correctness` view. This view returns a single distinct match count for each dictionary and corpus, confirming that all strategies reproduce the exact match set of the sequential baseline.


idp

**Ensino** que
te **conecta**